EWB CLOUD LABS

AWS VPC Peering

Private EC2 Connectivity Between Two Amazon VPCs

Project Report – Cloud Computing and Networking

Project title	AWS VPC Peering – EWB Cloud Labs
Project category	Cloud Computing and Networking
Student name	Ashrafunnisa Dambal
Organization	EWB Cloud Labs
AWS Region	Asia Pacific (Mumbai) – ap-south-1
Lab period	30 – 31 October 2026
Completion date	31 October 2026 (cleanup completed)

AWS services used

Service / component	Role in this project
Amazon VPC (two VPCs)	Isolated private networks: ewb-vpc-a (10.10.0.0/16) and ewb-vpc-b (10.20.0.0/16)
Subnets and route tables	ewb-a-public, ewb-a-private, ewb-b-private and their route tables
Internet Gateway	ewb-igw-a: internet path for the public subnet in VPC A
VPC peering connection	ewb-a-b-peer: private connectivity between VPC A and VPC B
Security groups	Instance-level SSH and ICMP Echo Request rules
Amazon EC2 (3 instances)	ewb-bastion, ewb-app-a and ewb-app-b (t3.micro, Amazon Linux 2023)
OpenSSH client (ProxyJump)	Laptop access to the private instances through the bastion host

Confidentiality note: no private key contents, passwords or access keys appear in this report. In the screenshots, the AWS account ID, the bastion's public IP address, the laptop's user and host name, the SSH host-key fingerprint and other personal details are covered with black boxes; the screenshots are otherwise unchanged.

Table of Contents

1. Introduction **3**
1.1 Results at a glance 3
2. Project Objectives **4**
3. AWS Architecture Overview **5**
3.1 Resource inventory and network configuration summary 5
3.2 Core concepts used in this design 6
4. Network Design and CIDR Planning **7**
4.1 CIDR notation and address planning 7
4.2 Non-overlapping ranges 7
4.3 Route tables and destination CIDRs 7
4.4 Why routes are required in both directions 7
5. Implementation and Configuration **8**
5.1 VPC creation 8
5.2 Subnet creation 8
5.3 Internet Gateway configuration 9
5.4 Route table configuration 9
5.5 Security group configuration 13
5.6 EC2 instance deployment 14
5.7 VPC peering connection 15
5.8 Peering route configuration 16
5.9 Bastion host and SSH ProxyJump configuration 16
6. Connectivity Validation and Test Results **17**
6.1 Test A: Baseline ping before peering 17
6.2 Test B: App-A to App-B ping after peering 18
6.3 Test C: App-B to App-A ping 19
6.4 Test D: SSH access to App-B through the bastion 21
7. Controlled ICMP Failure and Recovery **22**
7.1 Failure test: ICMP rule removed 22
7.2 Recovery test: ICMP rule restored 23
7.3 Why this experiment validates the role of security groups 23
8. AWS Resource Cleanup Evidence **24**
8.1 Cleanup order 24
8.2 Cleanup screenshots 25
9. Challenges Faced and Troubleshooting **27**
9.1 Issues observed during the lab 27
9.2 Troubleshooting method 27
9.3 Key technical points 27
10. What I Learned **28**
11. Conclusion **28**
12. Evidence Appendix **29**
12.1 Evidence index 29
12.2 Supplementary screenshots 30
13. Final Submission Checklist **32**

1. Introduction

This report documents the design, configuration, validation and cleanup of a VPC peering lab completed as part of the EWB Cloud Labs AWS internship programme. Two Amazon Virtual Private Clouds (VPCs) with non-overlapping address ranges were created in the Asia Pacific (Mumbai) Region and connected through a VPC peering connection. A bastion host in VPC A provides SSH access from the student's laptop to the private EC2 instances in both VPCs by using SSH ProxyJump. ICMP Echo Request rules in the security groups are used to test private connectivity, and a controlled failure-and-recovery experiment shows the role of the security group rule.

The report follows the structure of the EWB Cloud Labs project guidance. Every configuration value and test result stated here comes from the student's own AWS console and terminal screenshots, which are included as numbered figures with evidence IDs (E1 to E14d).

1.1 Results at a glance

Area	Result	Evidence
VPCs and CIDR blocks	**COMPLETED – 10.10.0.0/16 and 10.20.0.0/16**	E1, E7
Subnets	**COMPLETED – 3 subnets, 3 associated route tables**	E2a, E2b, E4a, E4d, E4f
Internet Gateway	**COMPLETED – ewb-igw-a attached to VPC A**	E3
Route tables	**COMPLETED – peering routes in both VPCs**	E4a to E4g
Security groups	**COMPLETED – SSH and ICMP Echo Request rules**	E5a, E5b
EC2 instances	**COMPLETED – bastion, App-A, App-B**	E6a, E6b, E14a
VPC peering connection	**COMPLETED – Status Active**	E7
Baseline ping (before peering)	**EXPECTED FAILURE – 100% packet loss**	E8
App-A to App-B ping	**PASS – 4/4 replies, 0% loss**	E13 (E9)
App-B to App-A ping	**PASS – 4/4 replies, 0% loss**	E10
SSH to App-B through bastion	**PASS – shell on ip-10-20-1-10**	E11
Controlled ICMP failure and recovery	**COMPLETED – 100% loss without rule, 0% loss after restore**	E12, E13
Resource cleanup	**COMPLETED – instances terminated, project VPCs deleted**	E14a to E14d

2. Project Objectives

The objectives below are taken from the project guidance; all were achieved.

#	Objective	Outcome and evidence
1	Build two isolated VPC networks with non-overlapping CIDR blocks.	Achieved: 10.10.0.0/16 and 10.20.0.0/16 (E1, E7).
2	Deploy private application instances (App-A in VPC A, App-B in VPC B).	Achieved: App-A 10.10.2.10 and App-B 10.20.1.10 in private subnets; App-B has no public IPv4 (E6b); both are reached only through the bastion (E11, E13).
3	Deploy a public bastion host in VPC A as the SSH jump host, with no NAT Gateway.	Achieved: bastion 10.10.1.10 in the public subnet (E6a); the private route tables have no default route and no NAT gateway is used (E4e, E4g).
4	Establish a VPC peering connection and add routes in both VPCs.	Achieved: Active peering (E7) and routes on both sides (E4b, E4e, E4g).
5	Validate private connectivity in both directions with ICMP.	Achieved: A to B (E13) and B to A (E10) both with 0% packet loss.
6	Reach the private instances through the bastion with SSH ProxyJump.	Achieved (E10, E11, E13).
7	Demonstrate the role of security group rules with a controlled ICMP failure and recovery.	Achieved: 100% loss with the rule removed (E12); 0% loss after restoring it (E13).
8	Clean up all resources to avoid ongoing charges.	Achieved (E14a to E14d).

3. AWS Architecture Overview

The design uses two VPCs in ap-south-1. VPC A contains a public subnet (bastion host) and a private subnet (App-A). VPC B contains one private subnet (App-B) and has no Internet Gateway. The peering connection ewb-a-b-peer carries private traffic between the VPCs, and each VPC has a route toward the other VPC's CIDR. The diagram reflects the deployed configuration shown in the evidence.

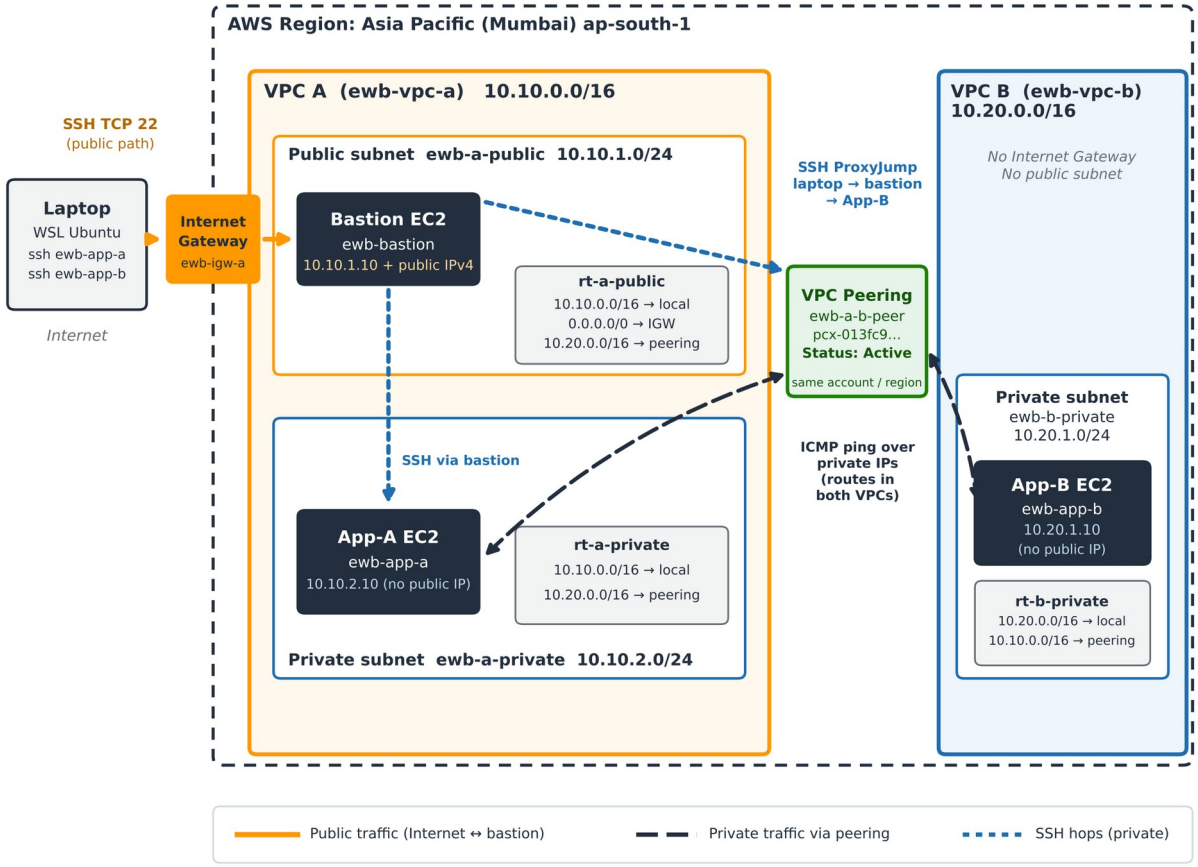


Figure 1 – Network architecture: VPC A, VPC B, VPC peering, bastion host and SSH paths (drawn from E1 to E7, E10 and E11).

Basis: CIDRs, IP addresses and route entries are those visible in the screenshots. The bastion sits in the public subnet (private IP 10.10.1.10 inside 10.10.1.0/24) and has an auto-assigned public IPv4 address. App-B's 'Last login ... from 10.10.1.10' (E11) shows the bastion-to-App-B SSH hop.

3.1 Resource inventory and network configuration summary

Resource	Name / ID	CIDR / IP / notes	Evidence
VPC A	ewb-vpc-a – vpc-0188875f43278c409	10.10.0.0/16	E1, E7
VPC B	ewb-vpc-b – vpc-0c0cc6e2ad636c65c	10.20.0.0/16	E1, E7
Public subnet A	ewb-a-public – subnet-0e5db0c5d1d1efdb1	10.10.1.0/24, ap-south-1a	E2a
Private subnet A	ewb-a-private – subnet-0ec00d74e319f7f0d	10.10.2.0/24, ap-south-1a	E4d, E14a
Private subnet B	ewb-b-private – subnet-0b6fcacf2d8ebea43	10.20.1.0/24, ap-south-1a	E2b
Internet Gateway	ewb-igw-a – igw-0f28e5c716f1edb0f	Attached to VPC A	E3
Route table (A public)	ewb-rt-a-public – rtb-017edf33f49d3d2d1	Associated with ewb-a-public	E4a, E4b
Route table (A private)	ewb-rt-a-private – rtb-098194110271492b2	Associated with ewb-a-private	E4d, E4e

Resource	Name / ID	CIDR / IP / notes	Evidence
Route table (B private)	ewb-rt-b-private – rtb-04c949616cfb9f67b	Associated with ewb-b-private	E4f, E4g
Peering connection	ewb-a-b-peer – pcx-013fc9ef046b2c343	Requester VPC A, accepter VPC B, Active	E7
Bastion EC2	ewb-bastion – i-075f2f0410c5ecfdf	10.10.1.10; auto-assigned public IPv4; t3.micro	E6a
App-A EC2	ewb-app-a – i-050a7fd32f5151e6b	10.10.2.10; private subnet; t3.micro	E14a, E12, E13
App-B EC2	ewb-app-b – i-016c26ac18f2df963	10.20.1.10; no public IPv4; t3.micro	E6b
Security group (bastion)	ewb-sg-bastion – sg-0a60ed37ec183a988	VPC A; inbound SSH (TCP)	E5a
Security group (App-B)	ewb-sg-app-b – sg-01f0a286c6c5c5057	VPC B; inbound SSH 22 and ICMP Echo Request	E5b
Security group (App-A)	ewb-sg-app-a	VPC A; inbound SSH 22 and ICMP Echo Request	E10, E11

3.2 Core concepts used in this design

Amazon VPC and private network isolation

An Amazon VPC is a logically isolated virtual network inside an AWS Region. Resources in one VPC cannot talk to resources in another VPC, or to the internet, unless routing and gateways are explicitly configured. This isolation is why the two VPCs cannot reach each other until peering and routes are added.

Public and private subnets; Internet Gateway

A subnet is a range of addresses inside a VPC, tied to one Availability Zone. A subnet is public when its route table sends 0.0.0.0/0 to an Internet Gateway (IGW), the gateway that connects a VPC to the internet, and its instances have public IPv4 addresses. A private subnet has no such route. Here ewb-rt-a-public sends 0.0.0.0/0 to the IGW (E4b), while the private route tables (E4e, E4g) have no default route, so App-A and App-B have no internet path.

EC2 instances and the bastion host

Amazon EC2 provides virtual servers. A bastion host is a small, hardened instance in a public subnet that is the single, controlled entry point for administration of private instances. Only the bastion has a public address; App-A and App-B accept SSH from the bastion's private address.

VPC peering

VPC peering is a one-to-one networking connection that lets instances in two VPCs communicate using private IPv4 addresses as if they were on the same network. Traffic stays on the AWS network and does not cross an Internet Gateway. Peering is not transitive, it requires non-overlapping CIDR blocks, and it does not edit route tables or security groups automatically.

4. Network Design and CIDR Planning

4.1 CIDR notation and address planning

CIDR notation writes an address range as a base address and a prefix length. The prefix says how many leading bits identify the network; the rest identify hosts. A /16 leaves 16 host bits (65,536 addresses) and a /24 leaves 8 host bits (256 addresses). Subnets must lie inside the VPC CIDR and must not overlap each other.

VPC	VPC CIDR	Subnet	Subnet CIDR	Type	Addresses	Evidence
VPC A	10.10.0.0/16	ewb-a-public	10.10.1.0/24	Public	256 (251 usable)	E2a
VPC A	10.10.0.0/16	ewb-a-private	10.10.2.0/24	Private	256 (251 usable)	E4d
VPC B	10.20.0.0/16	ewb-b-private	10.20.1.0/24	Private	256 (251 usable)	E2b

AWS reserves five addresses in every subnet (network address, VPC router, DNS, reserved for future use, broadcast), leaving 251 usable. The subnet pages show 250 available addresses (E2a, E2b) because one instance (10.10.1.10 or 10.20.1.10) already occupies an address.

4.2 Non-overlapping ranges

10.10.0.0/16 covers 10.10.0.0 to 10.10.255.255 and 10.20.0.0/16 covers 10.20.0.0 to 10.20.255.255. The second octet differs, so the ranges share no address. This is mandatory for peering: if both VPCs used the same range, a route for the destination CIDR could not tell the local network from the remote one, and AWS would refuse the peering connection.

4.3 Route tables and destination CIDRs

A route table is a set of rules (destination CIDR to target) that the VPC router consults for each packet leaving a subnet. The most specific matching destination wins. Every route table has an implicit local route for its own VPC CIDR; additional routes point other destinations to gateways or peering connections.

Route table (subnet)	Destination	Target	Status	Evidence
ewb-rt-a-public (ewb-a-public)	10.10.0.0/16	local	**Active**	E4b
ewb-rt-a-public (ewb-a-public)	0.0.0.0/0	igw-0f28e5c716f1edb0f (ewb-igw-a)	**Active**	E4b, E3
ewb-rt-a-public (ewb-a-public)	10.20.0.0/16	pcx-013fc9ef046b2c343 (ewb-a-b-peer)	**Active**	E4b, E7
ewb-rt-a-private (ewb-a-private)	10.10.0.0/16	local	**Active**	E4e
ewb-rt-a-private (ewb-a-private)	10.20.0.0/16	pcx-013fc9ef046b2c343 (ewb-a-b-peer)	**Active**	E4e, E7
ewb-rt-b-private (ewb-b-private)	10.20.0.0/16	local	**Active**	E4g
ewb-rt-b-private (ewb-b-private)	10.10.0.0/16	pcx-013fc9ef046b2c343 (ewb-a-b-peer)	**Active**	E4g, E7

The route-table screenshots truncate the target IDs; the full IDs appear in E3 and E7.

4.4 Why routes are required in both directions

A ping needs two packets to cross the peering link: the Echo Request from the sender and the Echo Reply from the receiver. The sender's subnet needs a route to the destination VPC, and the receiver's subnet needs a route back to the sender's VPC. If either route is missing, one of the two packets is dropped and the ping fails even though the peering connection is Active. The deployed configuration has the route toward VPC B in the A route tables (E4b, E4e) and the route toward VPC A in the B route table (E4g).

5. Implementation and Configuration

This section records each build step, the resulting configuration seen in the console, and the screenshot that supports it.

5.1 VPC creation

Two VPCs were created: ewb-vpc-a (10.10.0.0/16) and ewb-vpc-b (10.20.0.0/16). The VPC list also shows an unnamed VPC (vpc-0640f30cf2946902c) that is not part of this project and was left untouched.

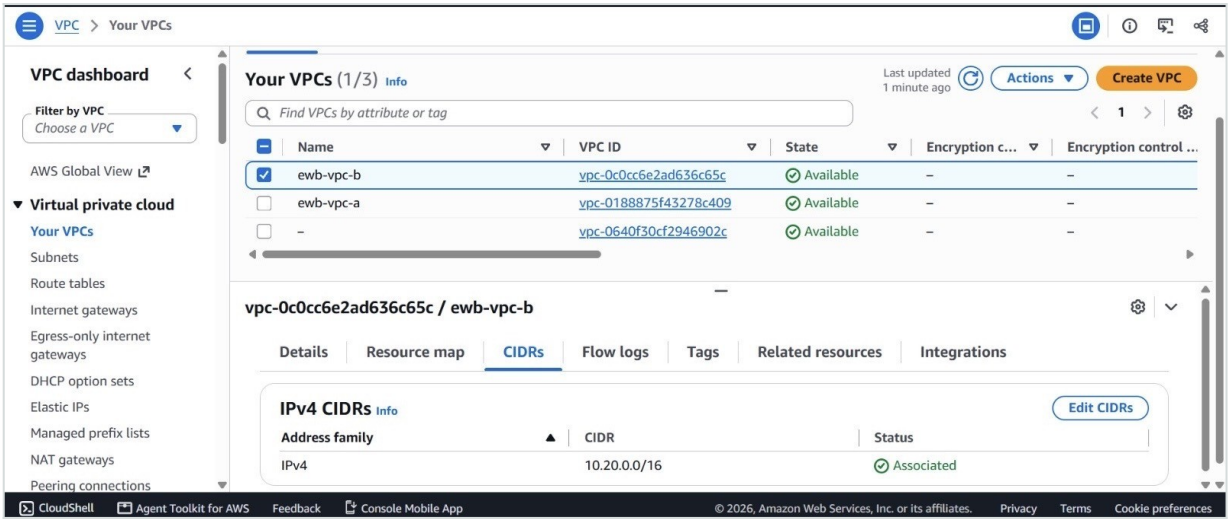


Figure 2 – E1: Your VPCs list with ewb-vpc-b selected, IPv4 CIDRs tab.

What it proves: VPC B (ewb-vpc-b, vpc-0c0cc6e2ad636c65c) is Available with IPv4 CIDR 10.20.0.0/16 (Associated). VPC A (ewb-vpc-a, vpc-0188875f43278c409) is listed as Available; its CIDR 10.10.0.0/16 is shown in Figure E7.

5.2 Subnet creation

Three subnets were created: ewb-a-public (10.10.1.0/24) and ewb-a-private (10.10.2.0/24) in VPC A, and ewb-b-private (10.20.1.0/24) in VPC B, all in ap-south-1a. Subnet-level auto-assign public IPv4 is No; the bastion receives its public address as an instance setting.

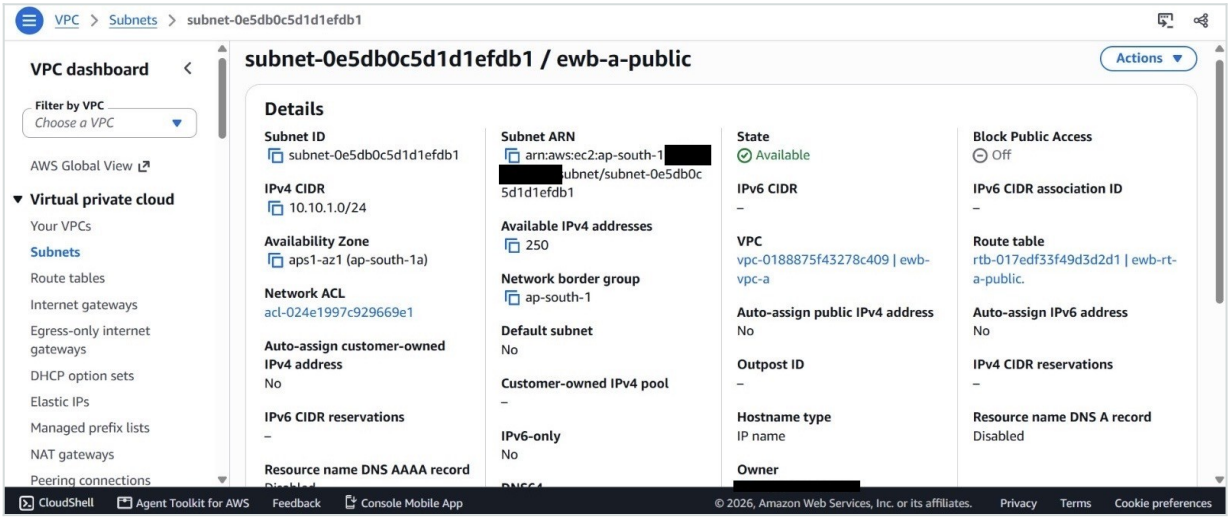


Figure 3 – E2a: Subnet details for ewb-a-public.

What it proves: Subnet subnet-0e5db0c5d1d1efdb1 has IPv4 CIDR 10.10.1.0/24, is Available in ap-south-1a (aps1-az1), belongs to ewb-vpc-a and uses route table ewb-rt-a-public. 250 IPv4 addresses are available (256, minus 5 reserved by AWS, minus the bastion's 10.10.1.10).

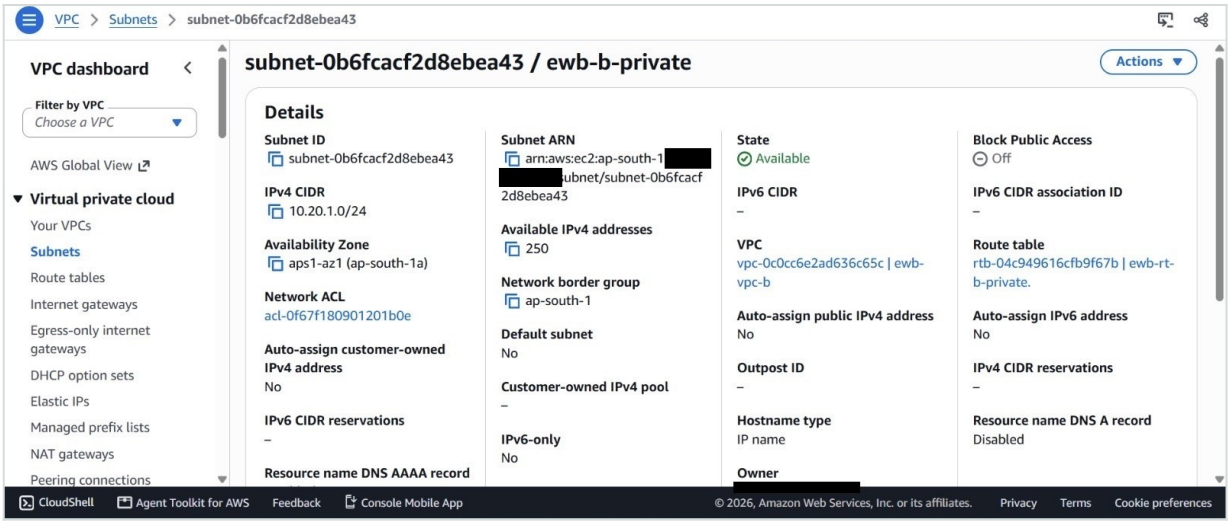


Figure 4 – E2b: Subnet details for ewb-b-private.

What it proves: Subnet subnet-0b6fcacf2d8ebea43 has IPv4 CIDR 10.20.1.0/24, is Available in ap-south-1a, belongs to ewb-vpc-b and uses route table ewb-rt-b-private. Auto-assign public IPv4 is No, consistent with a private subnet.

5.3 Internet Gateway configuration

An Internet Gateway, ewb-igw-a, was created and attached to VPC A only. It is the target of the 0.0.0.0/0 route in the public route table (E4b). VPC B has no Internet Gateway.

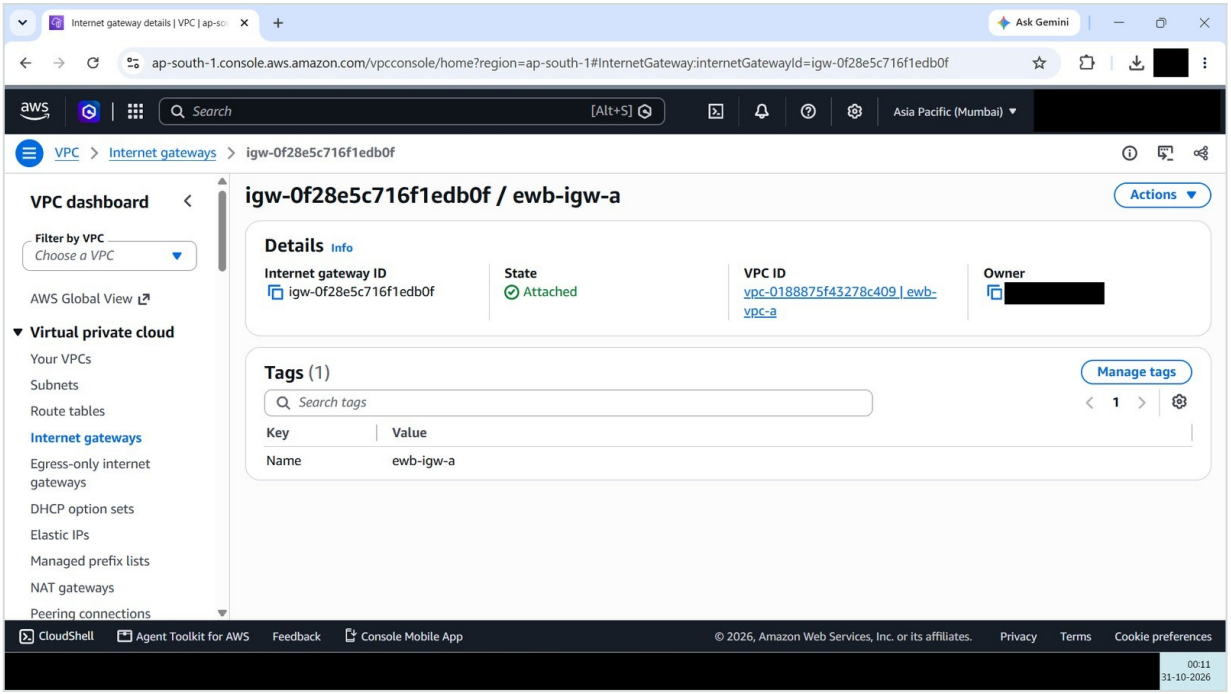


Figure 5 – E3: Internet gateway ewb-igw-a is attached to ewb-vpc-a.

What it proves: Internet gateway igw-0f28e5c716f1edb0f (Name tag ewb-igw-a) shows State Attached and VPC ID vpc-0188875f43278c409 (ewb-vpc-a). The console header shows the Asia Pacific (Mumbai) Region. Only VPC A has an Internet gateway.

5.4 Route table configuration

Three custom route tables are used. Each is explicitly associated with exactly one subnet, so the subnets do not depend on the VPC main route tables.

ewb-rt-a-public (VPC A, public subnet)

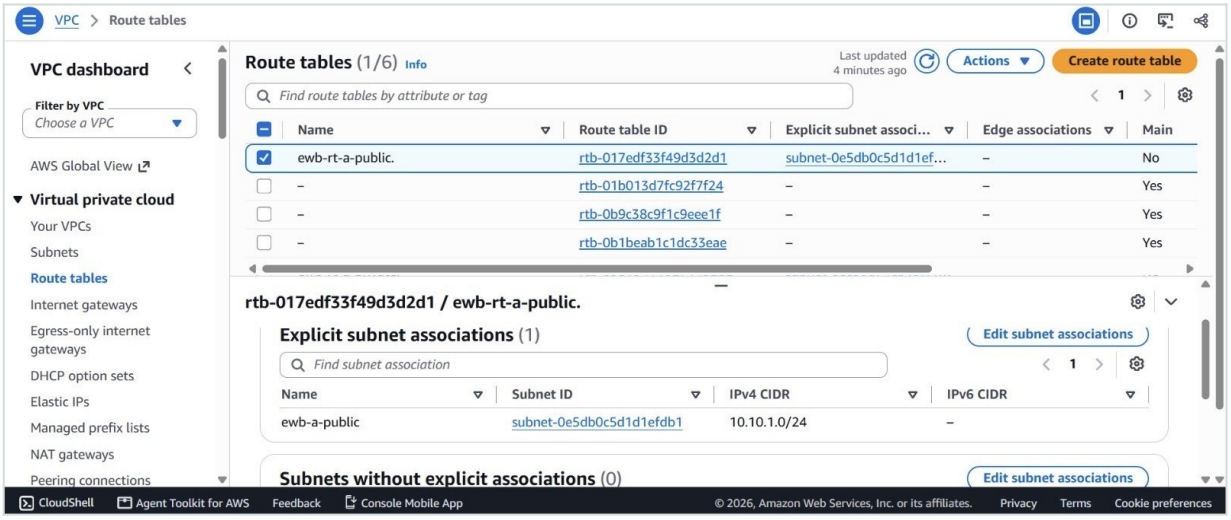


Figure 6 – E4a: Route table ewb-rt-a-public: explicit subnet association.

What it proves: Route table rtb-017edf33f49d3d2d1 (ewb-rt-a-public) is explicitly associated with subnet ewb-a-public (subnet-0e5db0c5d1d1efdb1, 10.10.1.0/24).

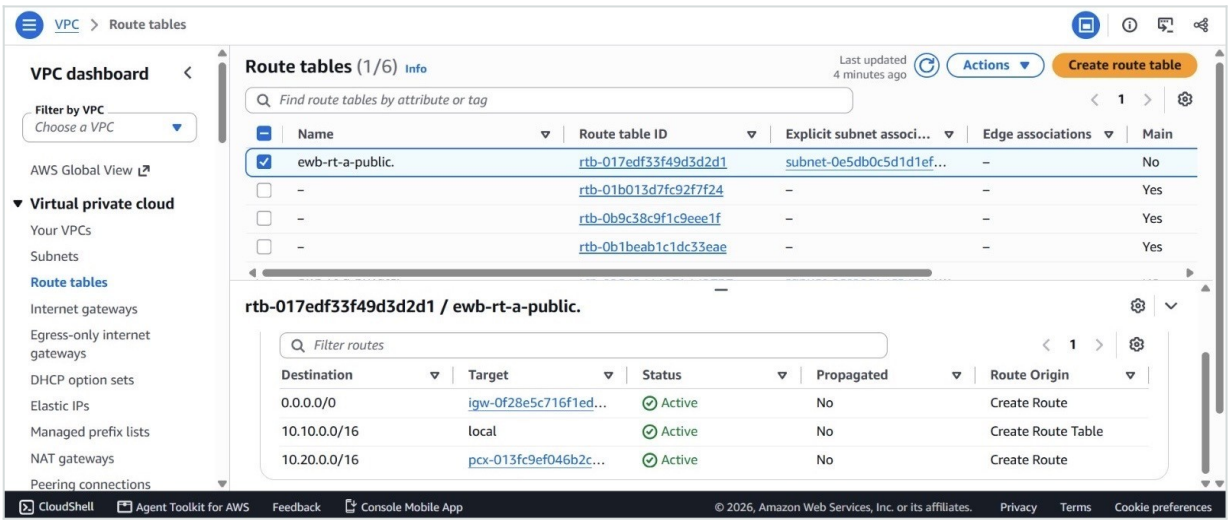


Figure 7 – E4b: Route table ewb-rt-a-public: routes.

What it proves: Three Active routes: 0.0.0.0/0 to the Internet gateway (igw-0f28e5c716f1ed…, the ID of ewb-igw-a in E3), 10.10.0.0/16 local, and 10.20.0.0/16 to the peering connection (pcx-013fc9ef046b2c…, the ID of ewb-a-b-peer in E7).

The table was first created with only the local and Internet Gateway routes; the peering route to 10.20.0.0/16 was added after the peering connection became Active.

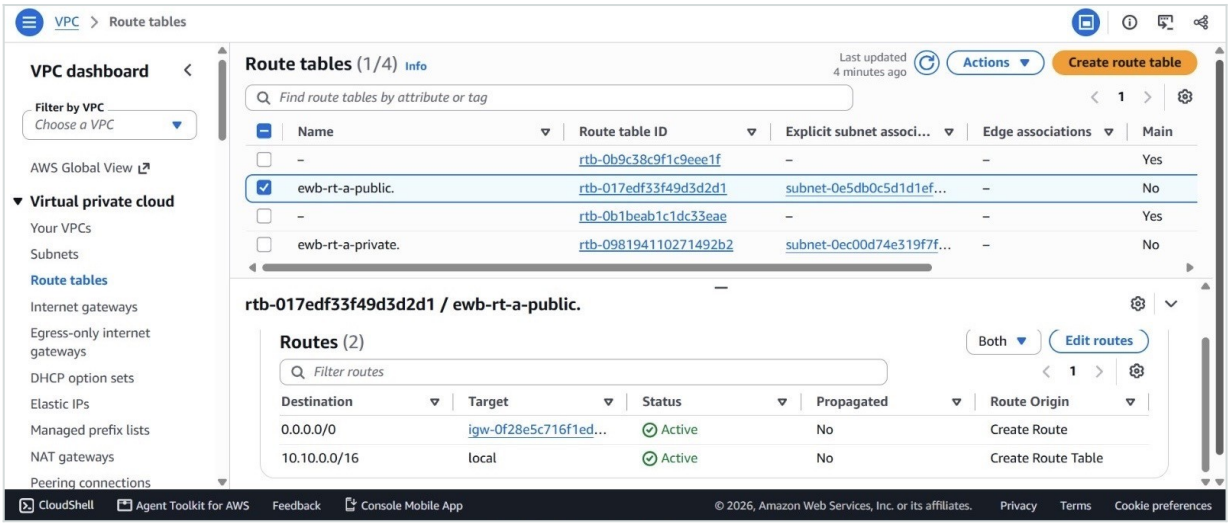


Figure 8 – E4c: Route table ewb-rt-a-public in its initial state (before the peering route was added).

What it proves: Only two routes are present: 0.0.0.0/0 to the Internet gateway and 10.10.0.0/16 local. The 10.20.0.0/16 peering route shown in E4b was added later.

ewb-rt-a-private (VPC A, private subnet)

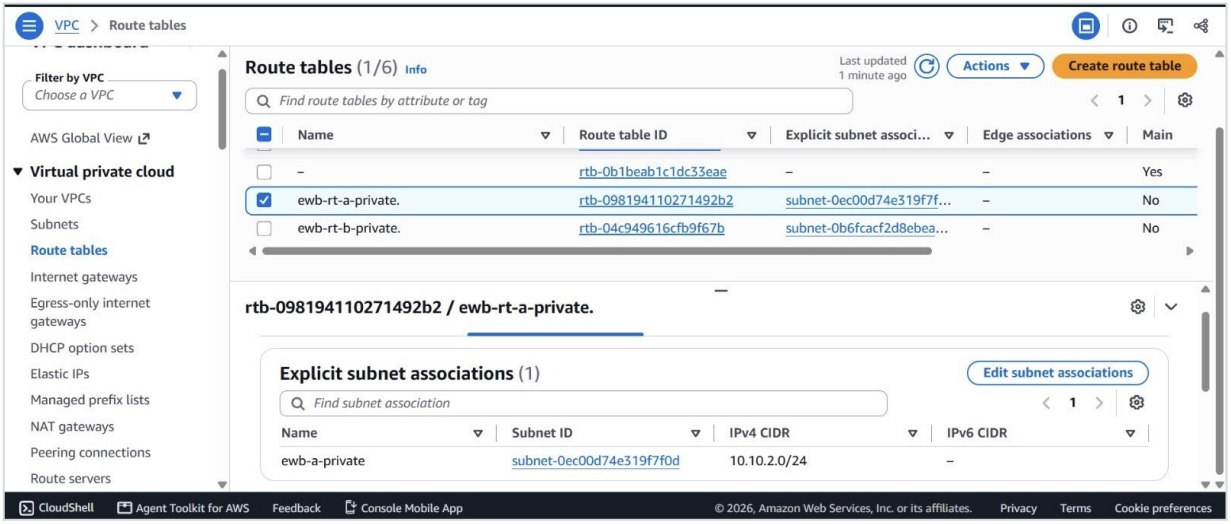


Figure 9 – E4d: Route table ewb-rt-a-private: explicit subnet association.

What it proves: Route table rtb-098194110271492b2 (ewb-rt-a-private) is explicitly associated with subnet ewb-a-private (subnet-0ec00d74e319f7f0d, 10.10.2.0/24).

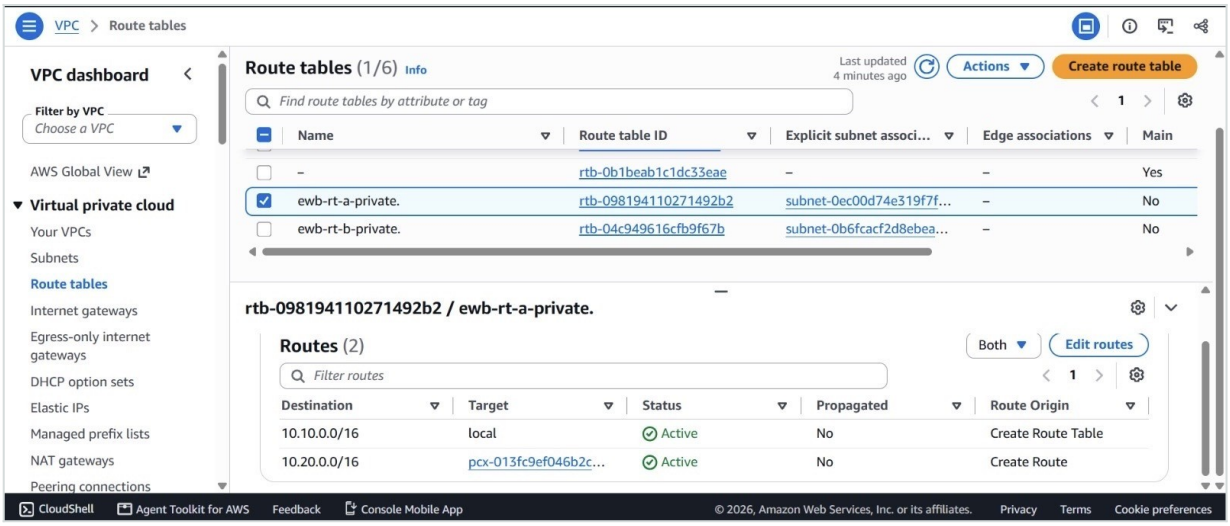


Figure 10 – E4e: Route table ewb-rt-a-private: routes.

What it proves: Two Active routes: 10.10.0.0/16 local and 10.20.0.0/16 to pcx-013fc9ef046b2c… (peering). There is no default route, so the subnet has no internet path.

ewb-rt-b-private (VPC B, private subnet)

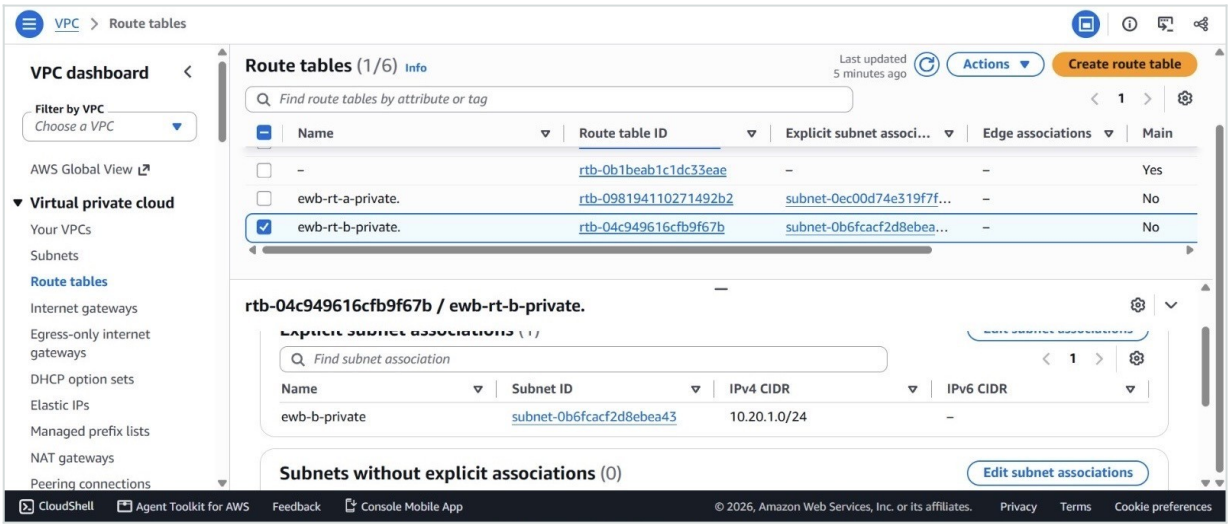


Figure 11 – E4f: Route table ewb-rt-b-private: explicit subnet association.

What it proves: Route table rtb-04c949616cfb9f67b (ewb-rt-b-private) is explicitly associated with subnet ewb-b-private (subnet-0b6fcacf2d8ebea43, 10.20.1.0/24).

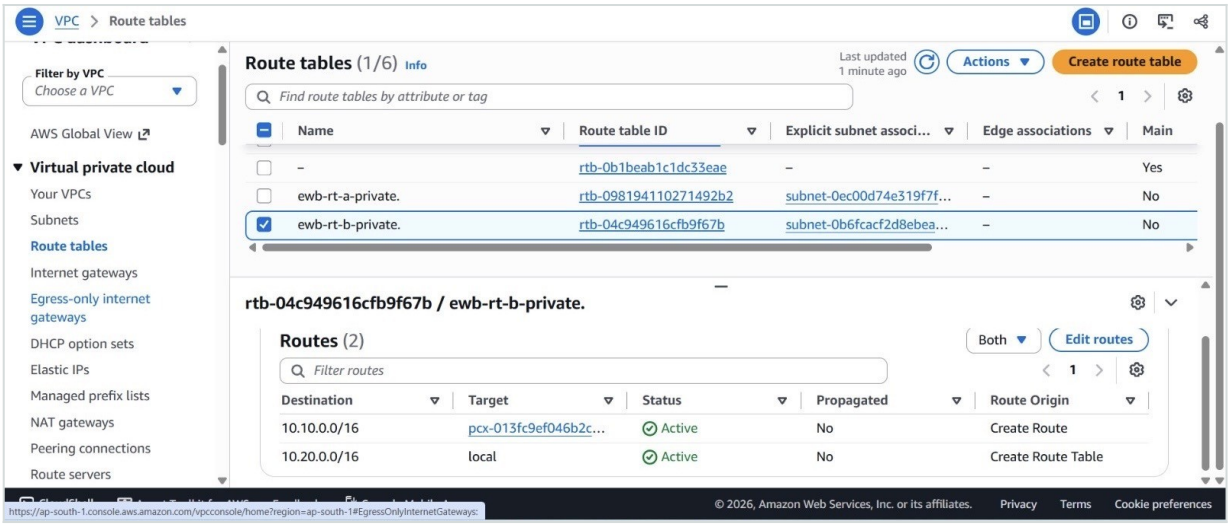


Figure 12 – E4g: Route table ewb-rt-b-private: routes.

What it proves: Two Active routes: 10.10.0.0/16 to pcx-013fc9ef046b2c… (peering) and 10.20.0.0/16 local. This is the return route that lets App-B reply to VPC A, so routing exists in both directions.

5.5 Security group configuration

Security groups are virtual firewalls attached to instances. They are **stateful**: if a request is allowed in, the reply is allowed out automatically, and the other way round, so no rule is needed for the return packet. Rules only allow traffic (there are no deny rules), and anything not explicitly allowed inbound is dropped. Outbound rules default to allow-all, which lets an instance send an ICMP Echo Reply or open an SSH connection without extra rules.

A ping is an ICMP Echo Request (type 8) from the sender and an ICMP Echo Reply (type 0) from the target. Because of statefulness, only the target's inbound rule for Echo Request is required; the reply is allowed automatically.

Security group	Inbound rules	Source restriction (project design)	Outbound	Evidence
ewb-sg-bastion	SSH TCP 22	Administrator's public IP /32	Default allow all	E5a
ewb-sg-app-a	SSH TCP 22; ICMP Echo Request	10.10.1.10/32 (bastion); 10.20.1.10/32 (App-B)	Default allow all	E10, E11
ewb-sg-app-b	SSH TCP 22; ICMP Echo Request	10.10.1.10/32 (bastion); 10.10.2.10/32 (App-A)	Default allow all	E5b, E13

Rules with a /32 source allow a single IP address, so each instance accepts traffic only from the one host that needs it (least privilege).

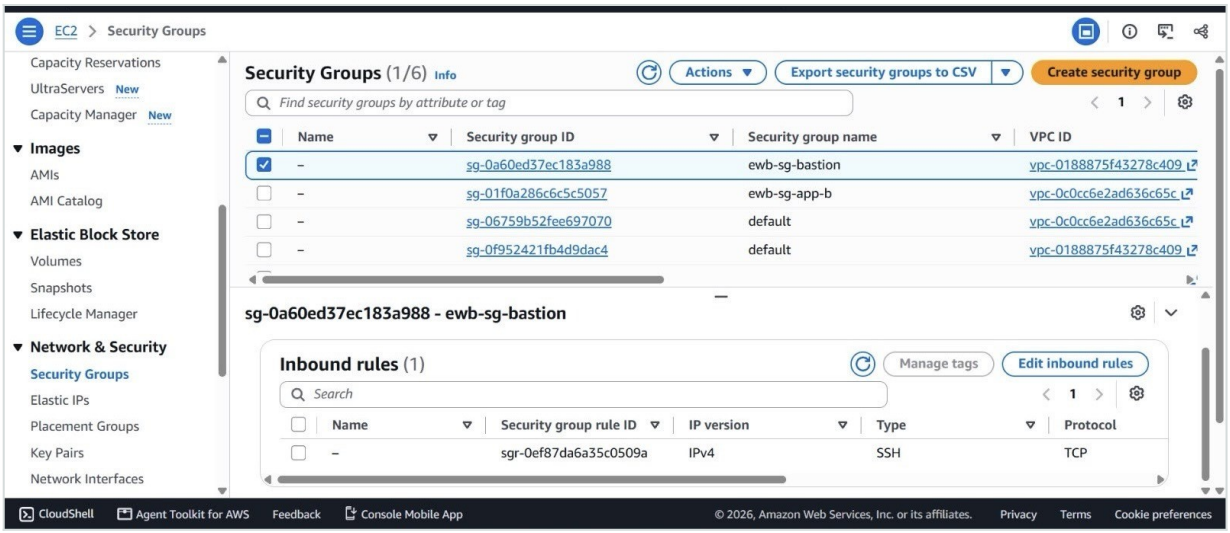


Figure 13 – E5a: Security group ewb-sg-bastion: inbound rules.

What it proves: Security group sg-0a60ed37ec183a988 (ewb-sg-bastion) belongs to vpc-0188875f43278c409 (VPC A) and has an inbound IPv4 SSH (TCP) rule, sgr-0ef87da6a35c0509a.

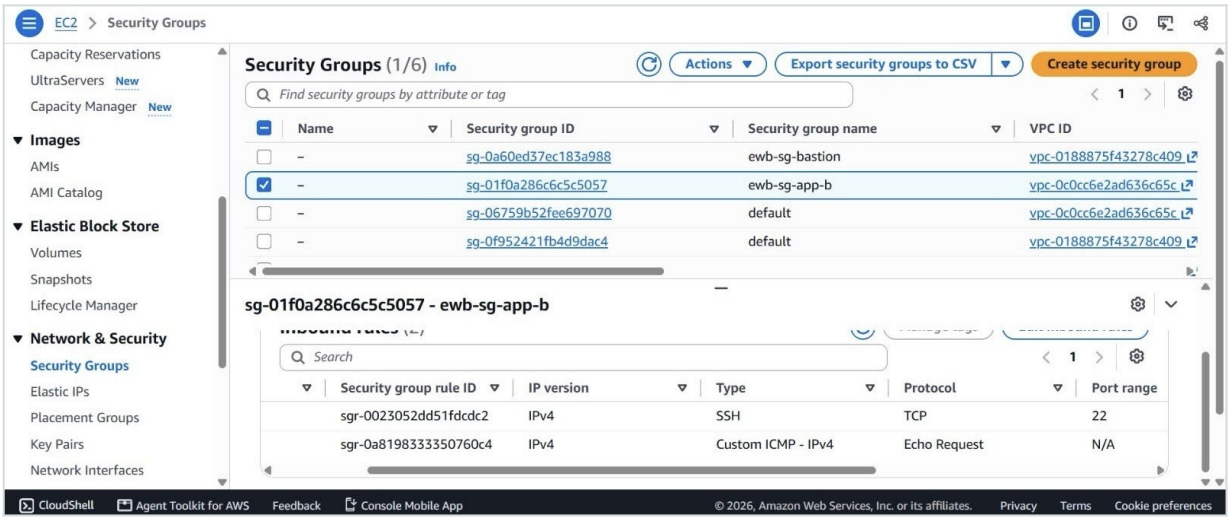


Figure 14 – E5b: Security group ewb-sg-app-b: inbound rules.

What it proves: Security group sg-01f0a286c6c5c5057 (ewb-sg-app-b) belongs to vpc-0c0cc6e2ad636c65c (VPC B) and has two inbound IPv4 rules: SSH TCP 22 (sgr-0023052dd51fdcdc2) and Custom ICMP - IPv4 Echo Request (sgr-0a8198333350760c4).

5.6 EC2 instance deployment

Three t3.micro instances running Amazon Linux 2023 were launched in ap-south-1a: ewb-bastion in the public subnet (private IP 10.10.1.10, auto-assigned public IPv4), ewb-app-a in the private subnet of VPC A (10.10.2.10) and ewb-app-b in VPC B (10.20.1.10, no public address). The login user is ec2-user.

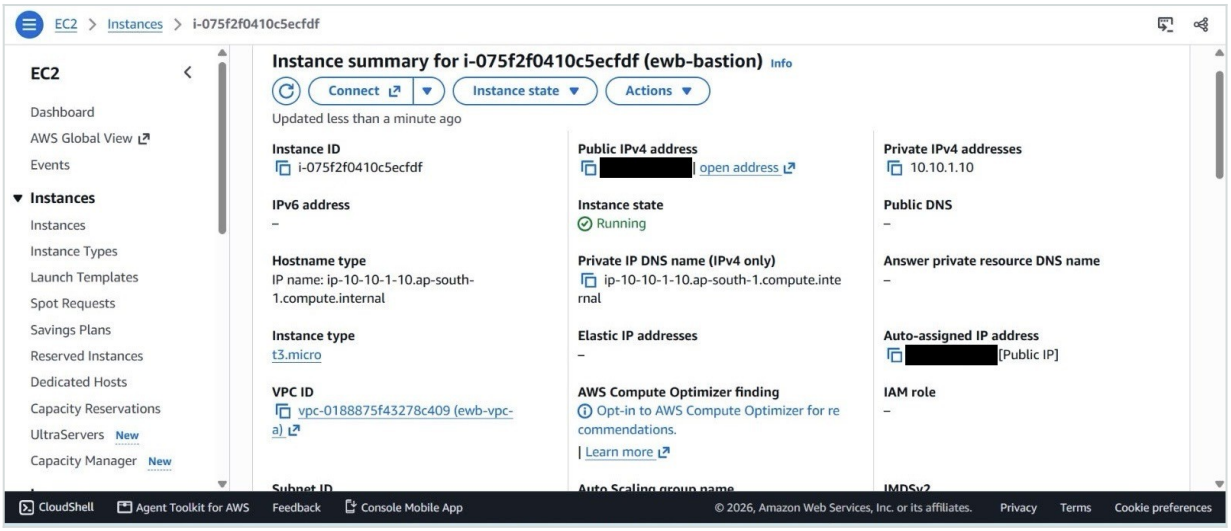


Figure 15 – E6a: EC2 instance summary for ewb-bastion.

What it proves: Instance i-075f2f0410c5ecfdf (ewb-bastion), t3.micro, Running, in ewb-vpc-a, private IPv4 10.10.1.10 and an auto-assigned public IPv4 address (masked in the screenshot). No Elastic IP is associated.

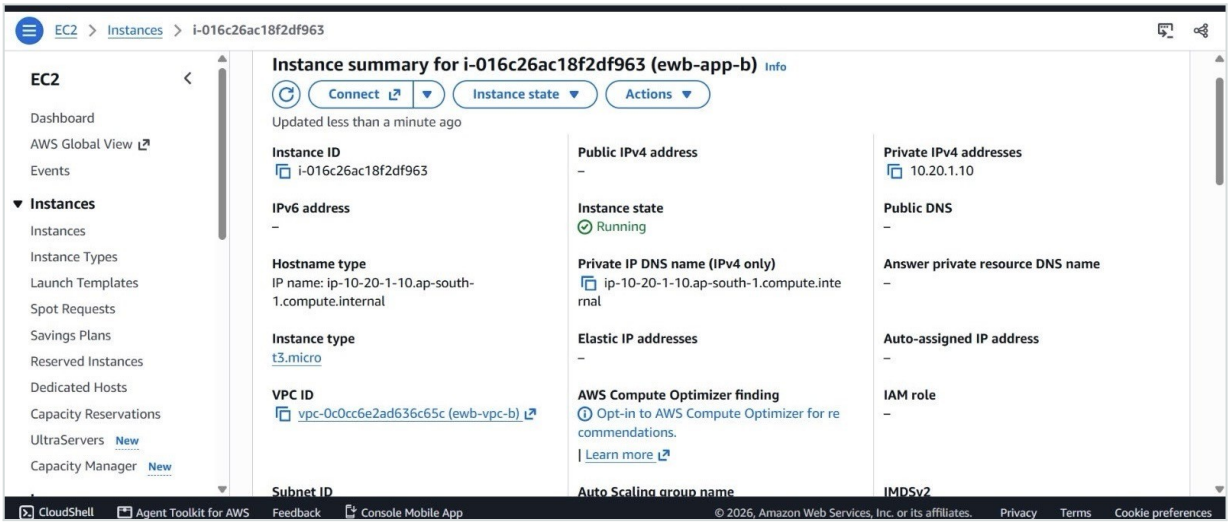


Figure 16 – E6b: EC2 instance summary for ewb-app-b.

What it proves: Instance i-016c26ac18f2df963 (ewb-app-b), t3.micro, Running, in ewb-vpc-b, private IPv4 10.20.1.10 with no public IPv4 address, no public DNS and no Elastic IP. App-B is not directly reachable from the internet.

5.7 VPC peering connection

The peering connection ewb-a-b-peer was requested from VPC A (requester) to VPC B (accepter), both in the same account and Region, and is in the Active state. Active means the request was accepted; it does not by itself allow traffic, which also needs routes and security group rules.

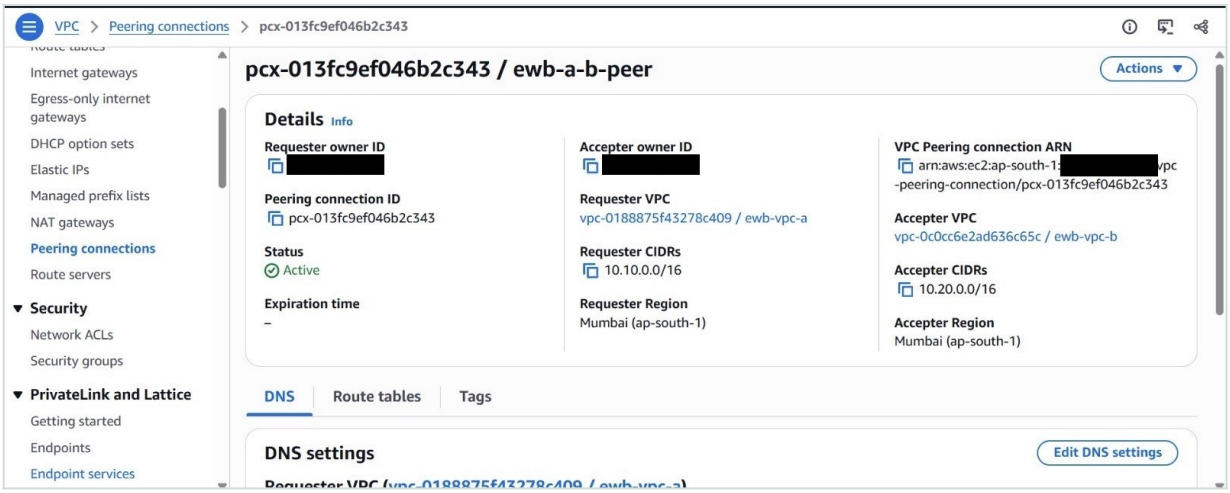


Figure 17 – E7: Peering connection ewb-a-b-peer (pcx-013fc9ef046b2c343).

What it proves: Status Active. Requester VPC is vpc-0188875f43278c409 (ewb-vpc-a, 10.10.0.0/16); accepter VPC is vpc-0c0cc6e2ad636c65c (ewb-vpc-b, 10.20.0.0/16). Both are in Mumbai (ap-south-1) and in the same account (the owner IDs are masked).

5.8 Peering route configuration

The peering routes are the 10.20.0.0/16 entries in ewb-rt-a-public (E4b) and ewb-rt-a-private (E4e) and the 10.10.0.0/16 entry in ewb-rt-b-private (E4g). All show Status Active and Route Origin 'Create Route', meaning they were added manually rather than propagated.

5.9 Bastion host and SSH ProxyJump configuration

SSH ProxyJump (the ProxyJump option, or ssh -J) tells the local OpenSSH client to open a connection to a jump host first and then tunnel a second SSH connection through it to the final target. The laptop's private key stays on the laptop; the bastion only forwards the encrypted stream, and the target instance needs no public address. With a Host alias, the whole path is reached with one command such as ssh ewb-app-b.

The aliases ewb-app-a and ewb-app-b on the laptop use the private IP of each instance as HostName and the bastion as the jump host. A representative configuration is shown below (key path omitted; the bastion alias name and its HostName placeholder are illustrative):

```
Host ewb-bastion
    HostName <bastion public IPv4>
    User ec2-user
    IdentityFile <path to key file>

Host ewb-app-a
    HostName 10.10.2.10
    User ec2-user
    ProxyJump ewb-bastion
    IdentityFile <path to key file>

Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
    ProxyJump ewb-bastion
    IdentityFile <path to key file>
```

Evidence that the setup works as described: (1) on the first connection OpenSSH reports the host as '10.20.1.10 (<no hostip for proxy command>)', showing the private IP was reached through a proxy hop (E10); (2) App-B's 'Last login ... from 10.10.1.10' shows the connection came from the bastion's private address (E11); (3) the hostnames ip-10-10-2-10 and ip-10-20-1-10.ap-south-1.compute.internal confirm which instance each session reached. No key contents are shown anywhere in this report.

6. Connectivity Validation and Test Results

ICMP, the protocol behind ping, is used for validation. The sender emits an ICMP Echo Request; the target replies with an Echo Reply. A reply proves routing in both directions and that the target's security group allows the request. A test is marked PASS only where the terminal output shows replies.

Test	Source to destination	Result	Evidence
A. Baseline ping	App-A (10.10.2.10) to 10.20.1.10	**EXPECTED FAILURE – 100% loss**	E8
B. App-A to App-B	App-A to 10.20.1.10	**PASS – 4/4 received, 0% loss**	E13 (E9)
C. App-B to App-A	App-B (10.20.1.10) to 10.10.2.10	**PASS – 4/4 received, 0% loss**	E10
D. SSH via bastion	Laptop to App-B (ssh ewb-app-b)	**PASS – shell on ip-10-20-1-10**	E11

6.1 Test A: Baseline ping before peering

Test objective	Show that App-A cannot reach App-B before the peering connection and routes exist.
Source	App-A (ewb-app-a), private IP 10.10.2.10 (prompt ec2-user@ip-10-10-2-10)
Destination	App-B (ewb-app-b), 10.20.1.10
Command	`ping -c 4 10.20.1.10`

Observed output	`[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10`
	`PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.`
	
	`--- 10.20.1.10 ping statistics ---`
	`4 packets transmitted, 0 received, 100% packet loss, time 3154ms`

Result	**EXPECTED FAILURE** No reply was received, as expected without a complete inter-VPC path.
Packet loss	100% (4 sent, 0 received)
Evidence	E8 (supplementary view E8-alt in the appendix)
Technical explanation	Before the peering connection and the routes existed, the VPC router had no path to 10.20.0.0/16 and dropped the Echo Request, so no reply returned and ping reported 100% loss. In the same capture, ssh ewb-app-b attempts to the private IP were interrupted with Ctrl-C because the connection could not complete without a path to VPC B.

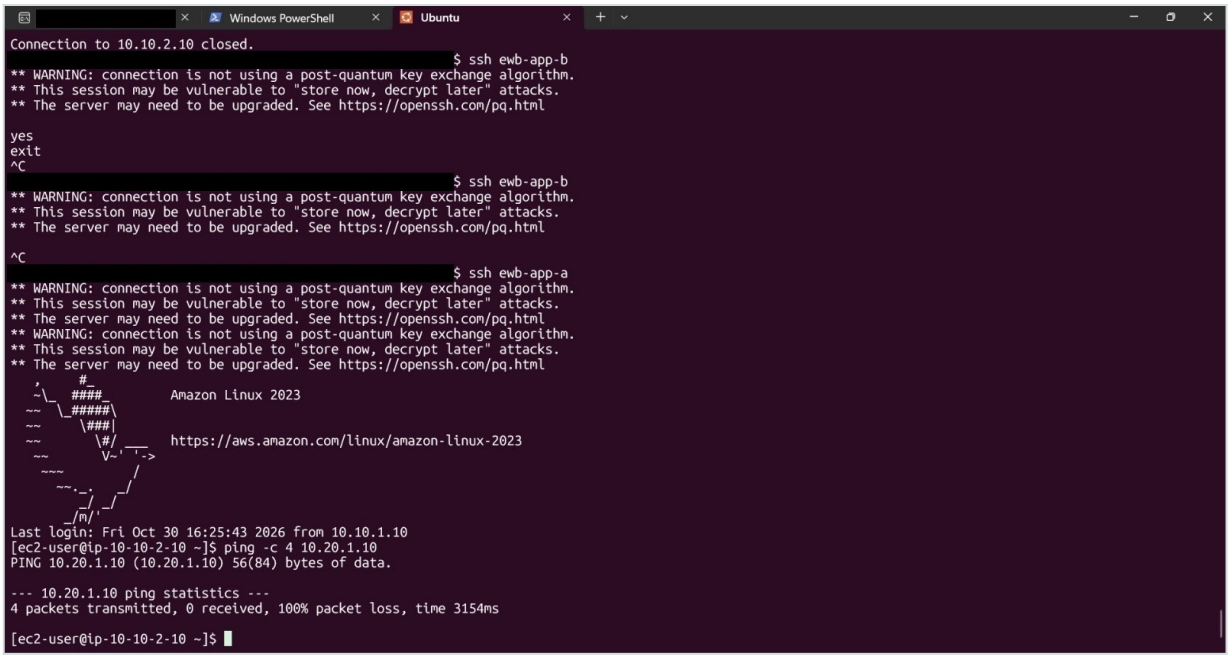


Figure 18 – E8: Terminal on App-A (ip-10-10-2-10): baseline ping -c 4 10.20.1.10 before peering.

What it proves: Captured before the peering connection and routes existed. The ping ends with '4 packets transmitted, 0 received, 100% packet loss, time 3154ms', as expected with no inter-VPC path. The lines above show two ssh ewb-app-b attempts that were interrupted with Ctrl-C before the peering was in place.

6.2 Test B: App-A to App-B ping after peering

Test objective	Show that App-A reaches App-B privately over the peering connection.
Source / destination	App-A (10.10.2.10) to App-B (10.20.1.10)
Command	`ping -c 4 10.20.1.10`

Observed output	`[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10`
	`PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.`
	`64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.198 ms`
	`64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.186 ms`
	`64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.202 ms`
	`64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.210 ms`
	
	`--- 10.20.1.10 ping statistics ---`
	`4 packets transmitted, 4 received, 0% packet loss, time 3114ms`
	`rtt min/avg/max/mdev = 0.186/0.199/0.210/0.008 ms`

Result	**PASS** All four Echo Requests were answered.
Packet loss	0% (4 sent, 4 received); average round-trip 0.199 ms
Evidence	E13, which is also the App-A to App-B test (E9)
Technical explanation	Replies from the private address 10.20.1.10 show that the A to B routes (E4b, E4e), the B to A return route (E4g), the Active peering connection (E7) and the Echo Request rule on ewb-sg-app-b (E5b) all worked. The ttl of 127 reflects the one extra hop through the VPC router, and the sub-millisecond latency is typical of instances in the same Availability Zone (ap-south-1a).

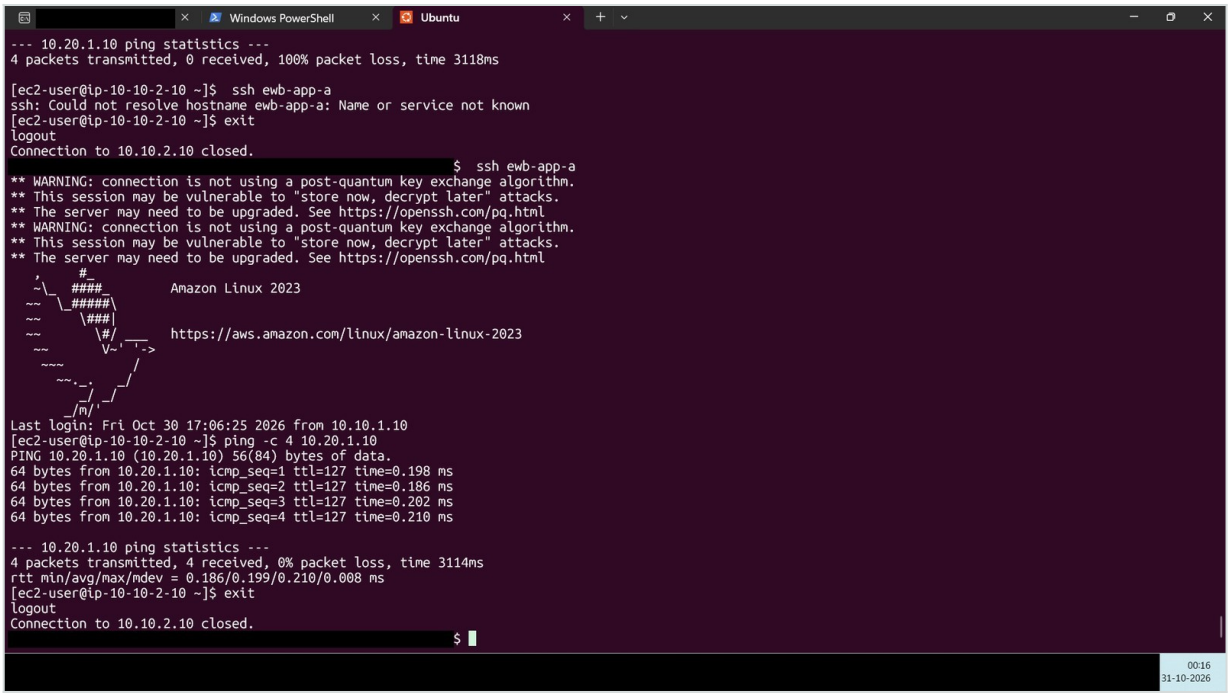


Figure 19 – E13: Terminal on App-A: ping -c 4 10.20.1.10 succeeds after the ICMP rule is restored (also the App-A to App-B test, E9).

What it proves: The failed 100% loss ping of E12 is visible at the top. After the rule was restored, a new session to App-A shows four replies from 10.20.1.10 (icmp_seq=1 to 4, ttl=127, 0.186 to 0.210 ms) and '4 packets transmitted, 4 received, 0% packet loss, time 3114ms'. The laptop clock at the bottom right reads 00:16 on 31-10-2026.

6.3 Test C: App-B to App-A ping

Test objective	Show that App-B reaches App-A privately over the peering connection.
Source	App-B (ewb-app-b), 10.20.1.10 (prompt ec2-user@ip-10-20-1-10, reached through the bastion)
Destination	App-A, 10.10.2.10
Command	`ping -c 4 10.10.2.10`

Observed output	`[ec2-user@ip-10-20-1-10 ~]$ ping -c 4 10.10.2.10`
	`PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.`
	`64 bytes from 10.10.2.10: icmp_seq=1 ttl=127 time=0.192 ms`
	`64 bytes from 10.10.2.10: icmp_seq=2 ttl=127 time=0.206 ms`
	`64 bytes from 10.10.2.10: icmp_seq=3 ttl=127 time=0.207 ms`
	`64 bytes from 10.10.2.10: icmp_seq=4 ttl=127 time=0.231 ms`
	
	`--- 10.10.2.10 ping statistics ---`
	`4 packets transmitted, 4 received, 0% packet loss, time 3081ms`
	`rtt min/avg/max/mdev = 0.192/0.209/0.231/0.014 ms`

Result	**PASS** All four Echo Requests were answered.
Packet loss	0% (4 sent, 4 received); average round-trip 0.209 ms
Evidence	E10
Technical explanation	Replies from 10.10.2.10 show that the B to A route (E4g), the return route in VPC A (E4e), the Active peering connection (E7) and the inbound ICMP rule on ewb-sg-app-a (source 10.20.1.10/32) all worked. Because security groups are stateful, no outbound rule was needed for the Echo Reply.

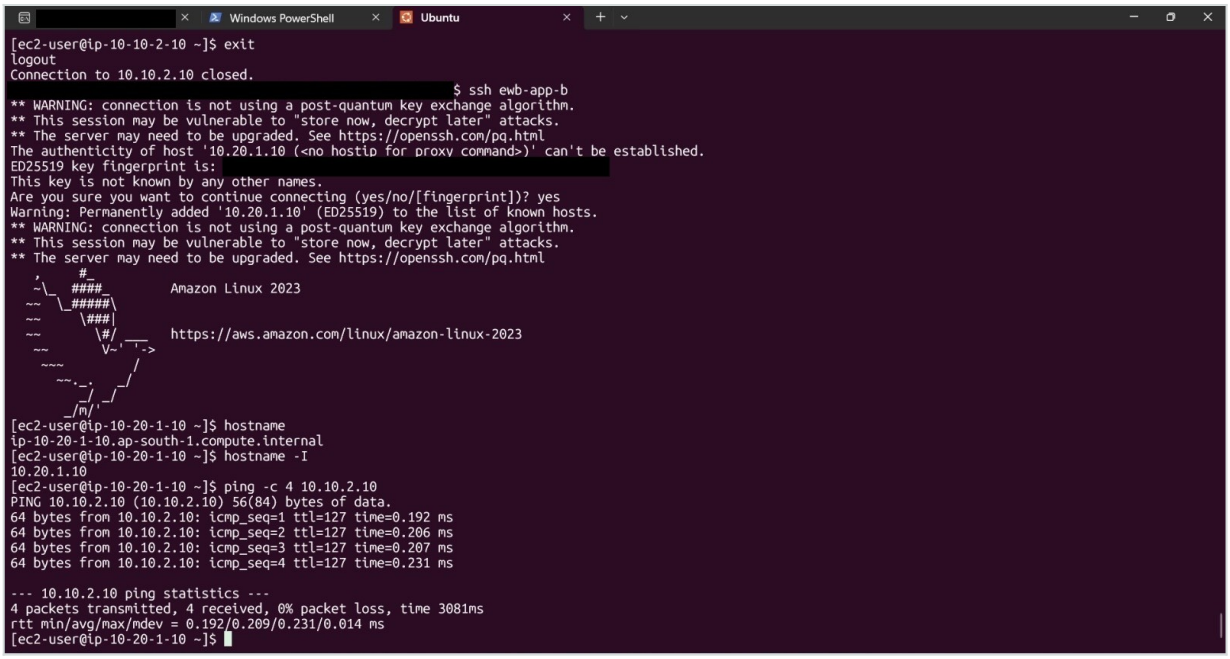


Figure 20 – E10: Terminal on App-B (ip-10-20-1-10): ping -c 4 10.10.2.10 succeeds.

What it proves: Four replies from 10.10.2.10 (icmp_seq=1 to 4, ttl=127, 0.192 to 0.231 ms) and '4 packets transmitted, 4 received, 0% packet loss, time 3081ms'. The screen also shows the first SSH connection to 10.20.1.10 through the bastion (host-key prompt) and hostname ip-10-20-1-10.ap-south-1.compute.internal / 10.20.1.10.

6.4 Test D: SSH access to App-B through the bastion

Test objective	Reach the private instance App-B from the laptop through the bastion host with a single command.
Source	Student's laptop (Ubuntu terminal)
Destination	App-B (ewb-app-b), 10.20.1.10, via the bastion (10.10.1.10)
Command	`ssh ewb-app-b`

Observed output	`$ ssh ewb-app-b`
	`[OpenSSH post-quantum key exchange warnings omitted]`
	`Amazon Linux 2023`
	`Last login: Fri Oct 30 16:56:48 2026 from 10.10.1.10`
	`[ec2-user@ip-10-20-1-10 ~]$ hostname`
	`ip-10-20-1-10.ap-south-1.compute.internal`
	`[ec2-user@ip-10-20-1-10 ~]$ hostname -I`
	`10.20.1.10`

Result	**PASS** An interactive shell on App-B was opened.
Packet loss	Not applicable
Evidence	E11 (the first connection, with the host-key prompt, is in E10)
Technical explanation	The client reached 10.20.1.10, which has no public address (E6b), through the bastion. App-B recorded the previous login as coming from 10.10.1.10, the bastion's private IP, proving the path laptop to bastion to App-B. The hop from the bastion to App-B crosses the VPC peering connection, so it also confirms TCP/22 reachability over the peering link and the SSH rule on ewb-sg-app-b. The transcript above is abbreviated from the screenshot.

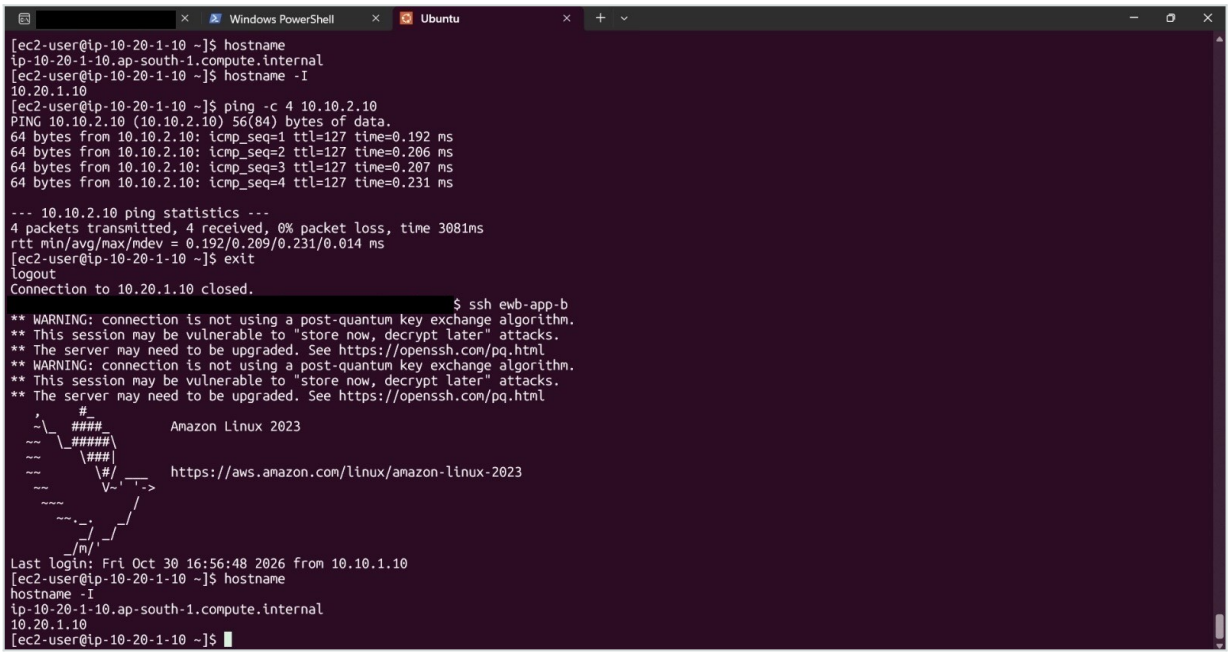


Figure 21 – E11: Terminal: ssh ewb-app-b from the laptop opens a shell on App-B.

What it proves: ssh ewb-app-b from the laptop reaches Amazon Linux 2023 at prompt ec2-user@ip-10-20-1-10; hostname returns ip-10-20-1-10.ap-south-1.compute.internal and hostname -I returns 10.20.1.10. 'Last login ... from 10.10.1.10' shows that the connection arrived from the bastion's private address.

7. Controlled ICMP Failure and Recovery

The controlled experiment checks that the inbound ICMP Echo Request rule on ewb-sg-app-b is what permits App-A to ping App-B. Only the security group rule was changed; the peering connection and the routes stayed as they were.

Step	Action	Result	Evidence
1. Original rule	ewb-sg-app-b has an inbound Custom ICMP - IPv4 Echo Request rule (plus SSH 22).	Rule present.	E5b
2. Rule removed	The Echo Request rule was deleted from ewb-sg-app-b.	Inbound ICMP to App-B no longer allowed.	Effect in E12
3. Failure test	From App-A: ping -c 4 10.20.1.10	4 sent, 0 received, 100% packet loss, time 3118ms.	E12
4. Rule restored	The Echo Request rule was added back (project design: source 10.10.2.10/32).	Inbound ICMP from App-A allowed again.	Effect in E13
5. Recovery test	From App-A: ping -c 4 10.20.1.10	4 sent, 4 received, 0% packet loss, time 3114ms, average 0.199 ms.	E13

7.1 Failure test: ICMP rule removed

Command	`ping -c 4 10.20.1.10  (from App-A, 10.10.2.10)`
Observed output	`--- 10.20.1.10 ping statistics ---` `4 packets transmitted, 0 received, 100% packet loss, time 3118ms`
Result	**FAILURE OBSERVED** 100% packet loss while the inbound Echo Request rule was absent.
Evidence	E12

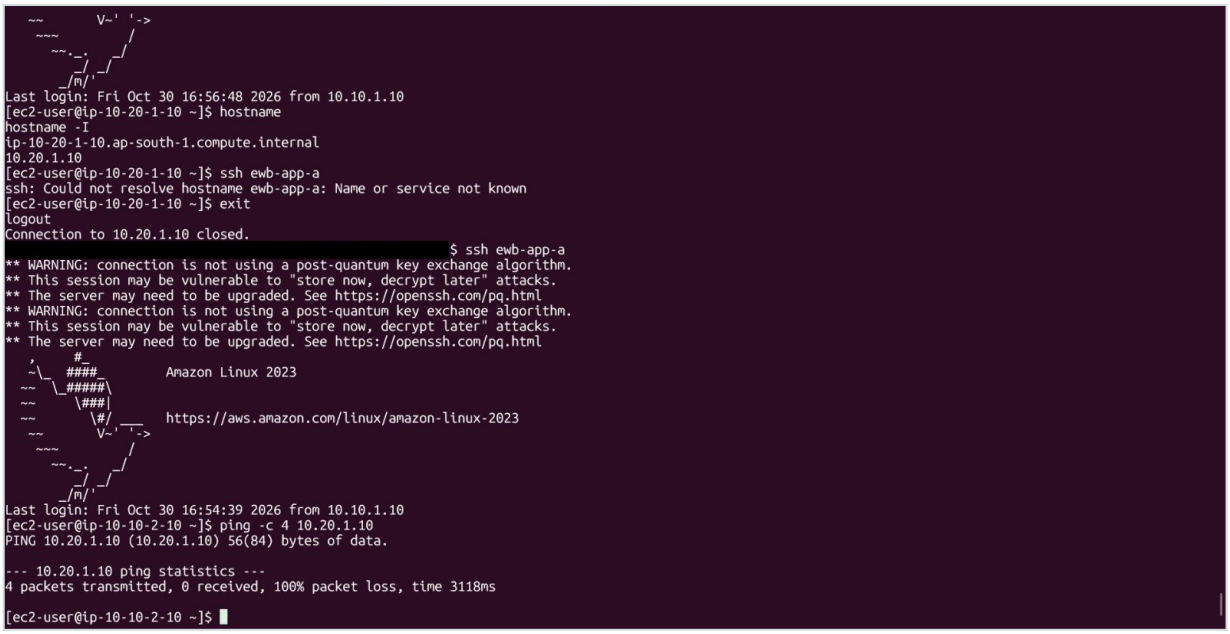


Figure 22 – E12: Terminal on App-A: ping -c 4 10.20.1.10 fails with 100% packet loss while the ICMP Echo Request rule is removed from ewb-sg-app-b.

What it proves: '4 packets transmitted, 0 received, 100% packet loss, time 3118ms'. The peering connection and routes were unchanged, so the failure is caused by the missing inbound ICMP rule. Earlier lines show 'ssh ewb-app-a' typed inside App-B failing with 'Could not resolve hostname', because the alias exists only in the laptop's SSH configuration.

7.2 Recovery test: ICMP rule restored

Command	`ping -c 4 10.20.1.10  (from App-A, new session)`

Observed output	`64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.198 ms`
	`64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.186 ms`
	`64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.202 ms`
	`64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.210 ms`
	`4 packets transmitted, 4 received, 0% packet loss, time 3114ms`

Result	**PASS** Connectivity recovered once the rule was restored.
Evidence	E13 (Figure 19)

7.3 Why this experiment validates the role of security groups

Routing decides whether a packet can find a path; the security group decides whether the destination instance accepts it. Removing only the security group rule leaves the peering connection and the routes untouched, so the change from successful pings to 100% loss is caused by the rule alone. When the rule is restored and the ping works again, that isolates the security group as the controlling element. Because security groups are stateful, the Echo Reply needs no outbound rule on App-B; only the inbound request rule matters. Restricting the source to 10.10.2.10/32 also shows least privilege: only App-A, not the whole VPC A range, may ping App-B.

8. AWS Resource Cleanup Evidence

After all evidence was captured, the project resources were removed to avoid ongoing charges. EC2 instances, attached EBS volumes and snapshots, and Elastic IP addresses that are not released all keep incurring cost, so they are removed first, followed by the networking resources in dependency order. The one VPC, subnet and main route table that remain were already present before the project and are not part of it (E14b, E14c, E14d).

Resource / task	Status	Evidence
EC2 instances ewb-bastion, ewb-app-a, ewb-app-b terminated	**COMPLETED**	E14a (all three Terminated)
EBS volumes and snapshots checked	**COMPLETED**	No project volumes or snapshots remain (student-confirmed)
Elastic IP addresses	**COMPLETED**	None allocated (E6a, E6b show no Elastic IP)
VPC peering connection and peering routes deleted	**COMPLETED**	Routes removed with the route tables (E14b); peering deletion student-confirmed
Security groups deleted	**COMPLETED**	Student-confirmed; project VPCs removed (E14d)
Subnets deleted	**COMPLETED**	E14c
Custom route tables deleted	**COMPLETED**	E14b
Internet Gateway detached and deleted	**COMPLETED**	Student-confirmed; VPC A removed (E14d)
VPCs deleted	**COMPLETED**	E14d
Final resource check	**COMPLETED**	E14a to E14d, captured 31-10-2026, 00:24 to 00:25

8.1 Cleanup order

Resources are deleted in dependency order, because AWS refuses to delete a resource that other resources still use:

1. Terminate the three EC2 instances and wait for the Terminated state; check EBS volumes, snapshots and Elastic IPs.
2. Delete the peering routes and the peering connection ewb-a-b-peer.
3. Delete the custom security groups once no network interface uses them.
4. Delete the subnets, then the custom route tables.
5. Detach the Internet Gateway from ewb-vpc-a, then delete it.
6. Delete ewb-vpc-a and ewb-vpc-b, then check the final resource lists.

8.2 Cleanup screenshots

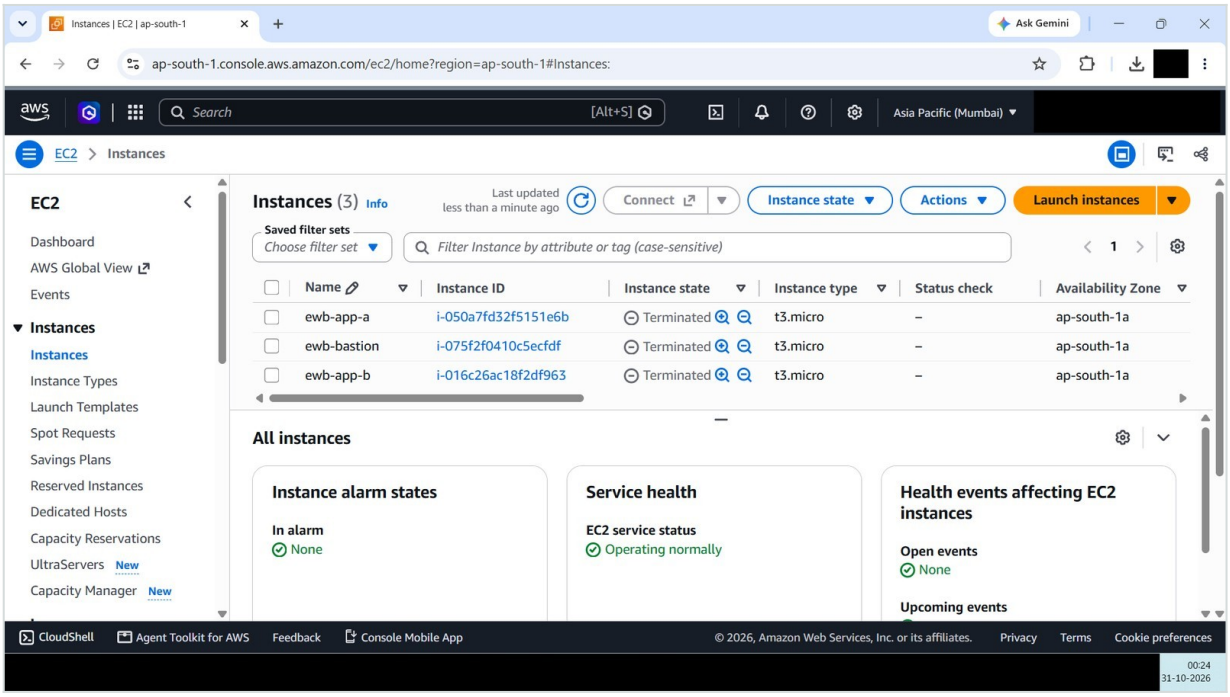


Figure 23 – E14a: EC2 Instances: ewb-app-a, ewb-bastion and ewb-app-b are all Terminated.

What it proves: ewb-app-a (i-050a7fd32f5151e6b), ewb-bastion (i-075f2f0410c5ecfdf) and ewb-app-b (i-016c26ac18f2df963), all t3.micro in ap-south-1a, show Instance state Terminated. Laptop clock: 00:24, 31-10-2026.

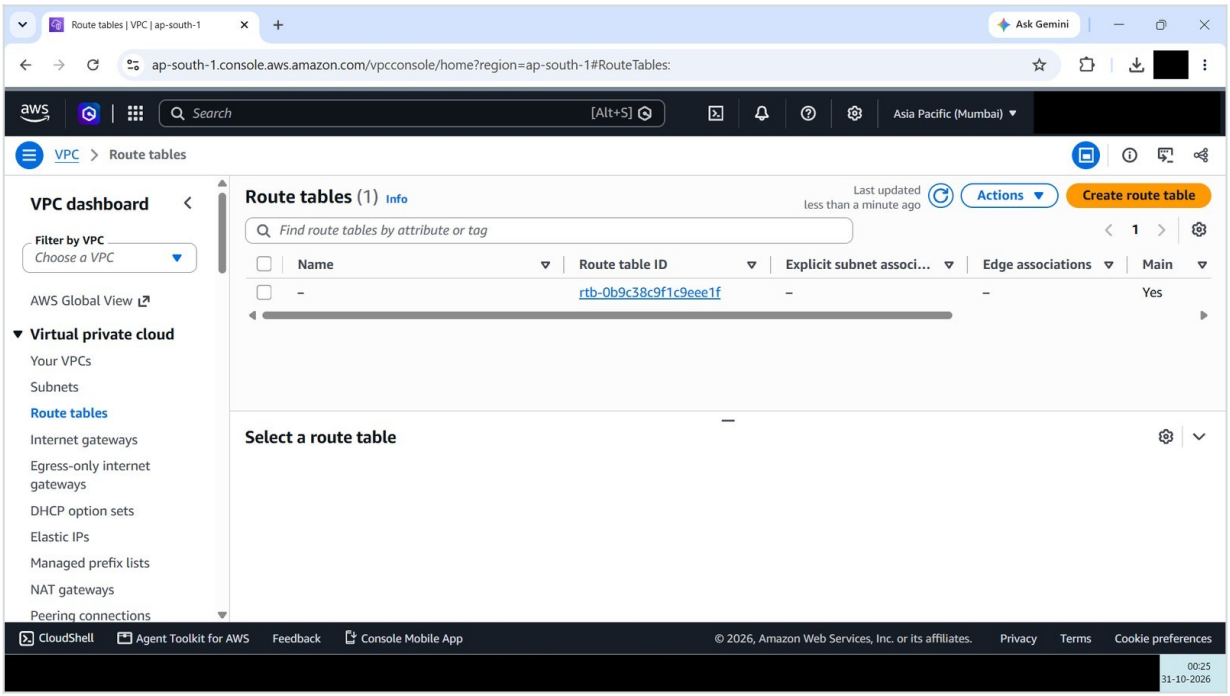


Figure 24 – E14b: Route tables: only one main route table remains.

What it proves: The list shows one route table (rtb-0b9c38c9f1c9eee1f, Main = Yes, no name, no explicit subnet association). ewb-rt-a-public, ewb-rt-a-private, ewb-rt-b-private and the main route tables of ewb-vpc-a and ewb-vpc-b no longer exist, so the peering routes are gone with them. Laptop clock: 00:25, 31-10-2026.

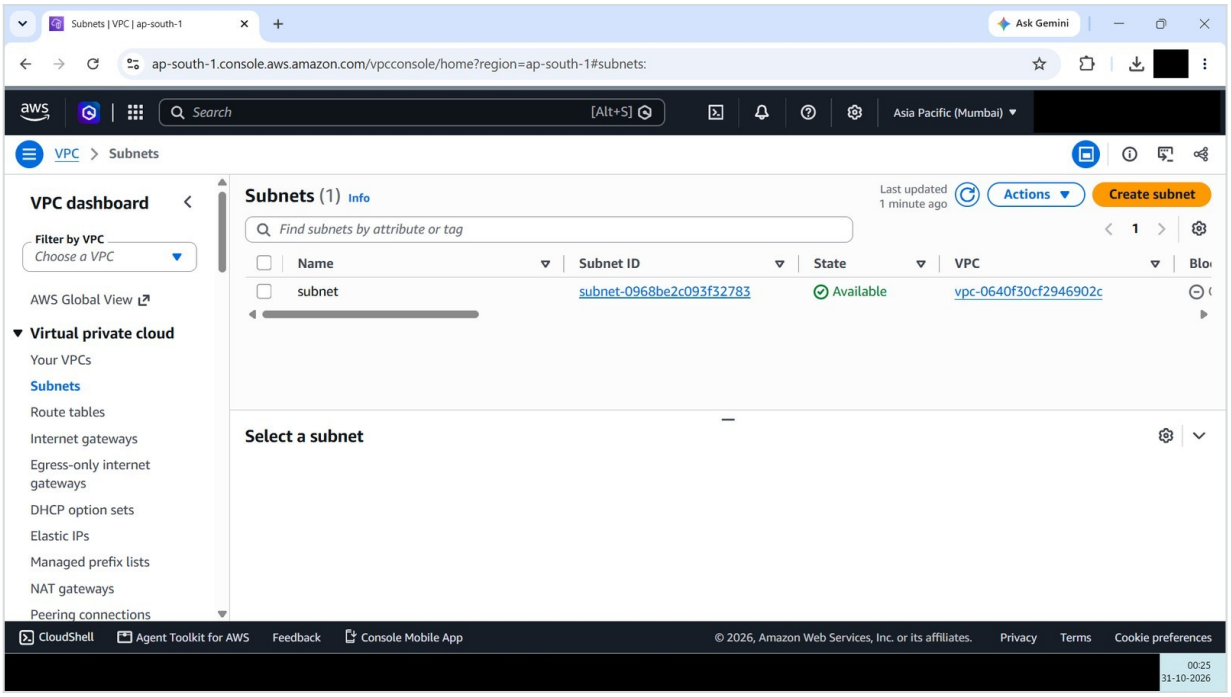


Figure 25 – E14c: Subnets: only one pre-existing subnet remains.

What it proves: The only subnet is an unnamed subnet (subnet-0968be2c093f32783) in vpc-0640f30cf2946902c, the pre-existing VPC that is not part of this project. ewb-a-public, ewb-a-private and ewb-b-private no longer exist. Laptop clock: 00:25, 31-10-2026.

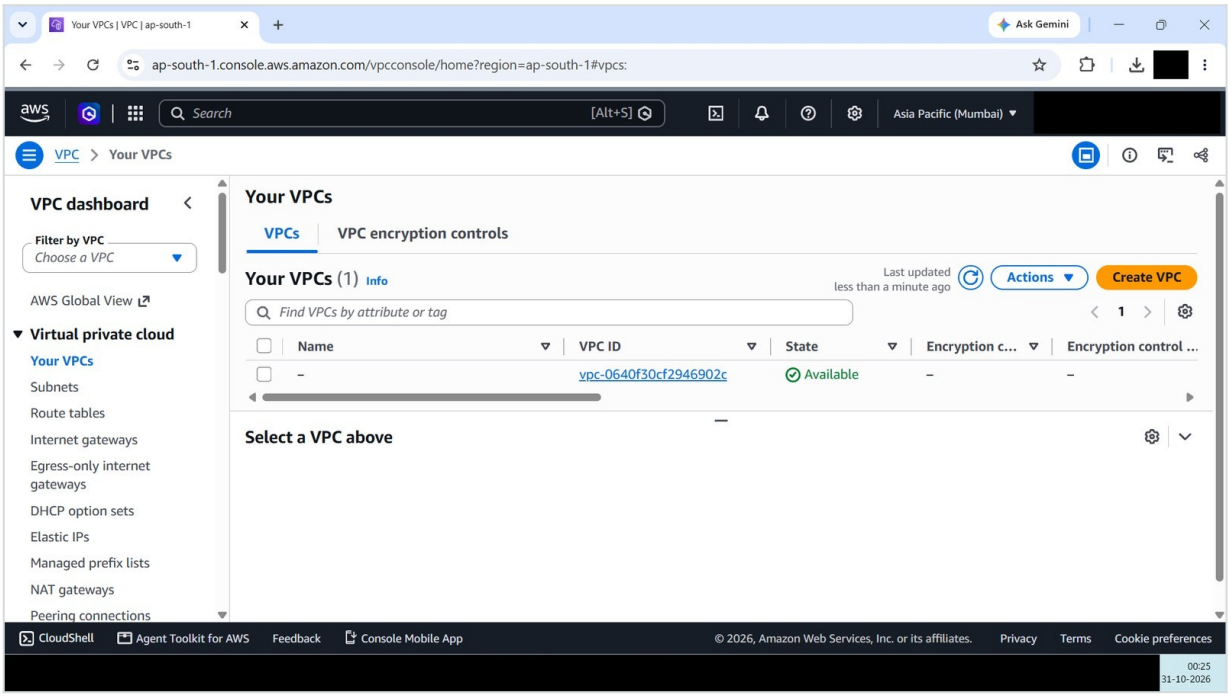


Figure 26 – E14d: Your VPCs: ewb-vpc-a and ewb-vpc-b no longer exist.

What it proves: Only the unnamed pre-existing VPC vpc-0640f30cf2946902c remains (it is also visible in E1). ewb-vpc-a and ewb-vpc-b have been deleted. Laptop clock: 00:25, 31-10-2026.

9. Challenges Faced and Troubleshooting

9.1 Issues observed during the lab

Observation	Evidence	Technical reading and outcome
App-A to App-B ping showed 100% packet loss before peering.	E8	Expected: no peering connection or routes existed. Solved by creating the peering connection and adding routes in both VPCs (E7, E4b, E4e, E4g); the ping then succeeded (E13).
ssh ewb-app-b attempts to the private instance hung and were interrupted with Ctrl-C before peering.	E8, E8-alt	The bastion had no path to VPC B yet. After peering and routes were in place, ssh ewb-app-b opened a shell (E11).
Ping failed with 100% loss after the ICMP rule was removed.	E12	Deliberate failure: the inbound Echo Request rule on ewb-sg-app-b was missing. Restoring the rule recovered the ping (E13).
'ssh ewb-app-a' typed inside an instance failed with 'Could not resolve hostname ewb-app-a'.	E12, E13	The alias is defined only in the laptop's SSH configuration; instances do not know it. Aliases are used from the laptop, or the private IP from inside the VPC.
First SSH connection to 10.20.1.10 asked to confirm the host key.	E10	Normal OpenSSH trust-on-first-use behaviour for a new host; it is not a fault.

9.2 Troubleshooting method

For a failed ping or SSH between VPCs, check in this order: (1) peering connection state is Active (E7); (2) a route for the destination CIDR in the source subnet's route table; (3) a route back to the source CIDR in the destination subnet's route table; (4) the destination security group allows the protocol from the source address (SSH TCP 22 or ICMP Echo Request); (5) outbound rules and network ACLs allow the traffic; (6) the instance is running and the target IP is correct. Compare each result with the screenshots above to find which layer is responsible.

9.3 Key technical points

Topic	Point to remember
CIDR blocks	The prefix length sets the size of the range; VPC ranges that will be peered must not overlap (10.10.0.0/16 and 10.20.0.0/16 here).
Bidirectional routing	An Active peering connection carries no traffic until routes exist in both VPCs (three peering routes in total here).
Failed pings	Work from the route tables, then the peering state, then the security groups to find which layer blocks the traffic.
Security group restrictions	/32 sources make each rule very specific, so a wrong source address blocks the traffic.
SSH through a bastion	Host aliases exist only on the laptop; ProxyJump lets one command reach a private instance without a public IP.
Controlled failure and recovery	Changing one control at a time shows which control causes a failure.
Resource cleanup	Dependencies force an order: instances, peering, security groups, subnets, route tables, gateway, VPCs.

10. What I Learned

- **CIDR design:** VPC and subnet CIDR blocks define private address spaces; VPCs that will be peered must not overlap (here 10.10.0.0/16 and 10.20.0.0/16).
- **Routing:** route tables decide whether traffic stays inside the VPC (local), goes to an Internet Gateway (0.0.0.0/0) or crosses a peering connection (10.20.0.0/16 and 10.10.0.0/16).
- **Peering:** peering needs routes in both directions and does not configure security groups automatically; an Active connection alone does not pass traffic.
- **Bastion and ProxyJump:** a bastion host and ProxyJump give controlled access to private instances that have no public IP.
- **Security groups:** rules for SSH and ICMP control what each instance accepts; because they are stateful, replies are allowed automatically.
- **Testing method:** a controlled failure and recovery test, with only one change at a time, isolates the control that matters.
- **Cost control:** cloud resources must be cleaned up and verified after a lab so that they do not accrue charges.

11. Conclusion

The project was completed as planned. Two VPCs with non-overlapping CIDR blocks (10.10.0.0/16 and 10.20.0.0/16) were created in ap-south-1, connected by an Active peering connection with routes in both directions, and validated with ICMP and SSH. The baseline ping before peering failed as expected (100% loss). After peering, App-A reached App-B and App-B reached App-A privately, each with 4 of 4 replies and 0% packet loss. App-B, which has no public IP address, was reached from the laptop through the bastion with ssh ewb-app-b.

The controlled experiment showed that the inbound ICMP Echo Request rule on ewb-sg-app-b controls the ping: removing it produced 100% packet loss and restoring it brought the ping back to 0% loss, while the peering connection and routes were unchanged. Finally, all three instances were terminated and the project VPCs, subnets and route tables were deleted, so no project resources remain to incur charges.

12. Evidence Appendix

12.1 Evidence index

Evidence ID	Screenshot	Shown in
E1	VPC B details and CIDR	Figure 2
E2a	Subnet ewb-a-public	Figure 3
E2b	Subnet ewb-b-private	Figure 4
E3	Internet gateway attached to VPC A	Figure 5
E4a	ewb-rt-a-public association	Figure 6
E4b	ewb-rt-a-public routes	Figure 7
E4c	ewb-rt-a-public initial routes	Figure 8
E4d	ewb-rt-a-private association	Figure 9
E4e	ewb-rt-a-private routes	Figure 10
E4f	ewb-rt-b-private association	Figure 11
E4g	ewb-rt-b-private routes	Figure 12
E5a	Security group ewb-sg-bastion	Figure 13
E5b	Security group ewb-sg-app-b	Figure 14
E6a	EC2 instance ewb-bastion	Figure 15
E6b	EC2 instance ewb-app-b	Figure 16
E7	Peering connection Active	Figure 17
E8	Baseline ping, 100% loss	Figure 18
E10	Ping App-B to App-A, success	Figure 20
E11	SSH session to App-B via bastion	Figure 21
E12	ICMP rule removed: ping failure	Figure 22
E13	ICMP rule restored: ping recovery (also E9, App-A to App-B)	Figure 19
E14a	Cleanup: instances terminated	Figure 23
E14b	Cleanup: route tables removed	Figure 24
E14c	Cleanup: subnets deleted	Figure 25
E14d	Cleanup: VPCs deleted	Figure 26
E4a-alt	ewb-rt-a-public association (alt.)	Appendix figure 27
E4e-alt	ewb-rt-a-private routes with console header	Appendix figure 28
E7-alt	Peering connection (alt.)	Appendix figure 29
E8-alt	App-A baseline ping in progress	Appendix figure 30

Screenshots are the originals from the AWS console and the terminal. The only change is that sensitive details (AWS account ID, public IP address, laptop user and host name, SSH host-key fingerprint, browser profile picture and taskbar) are covered with black boxes.

12.2 Supplementary screenshots

The screenshots below duplicate or extend items shown earlier and are kept for completeness.

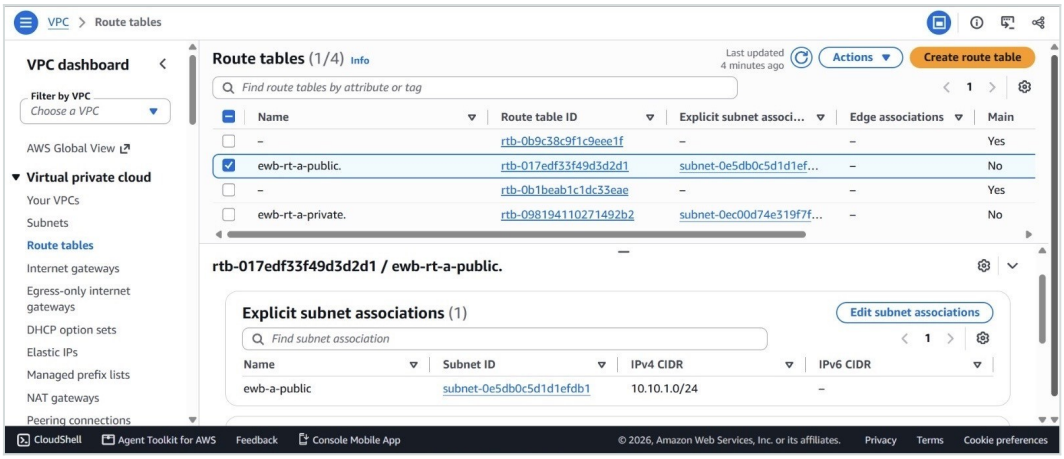


Figure 27 – E4a-alt: Route table ewb-rt-a-public: explicit subnet association, 'Route tables (1/4)' view.

What it proves: Same association as E4a (ewb-a-public, 10.10.1.0/24), captured in the earlier view.

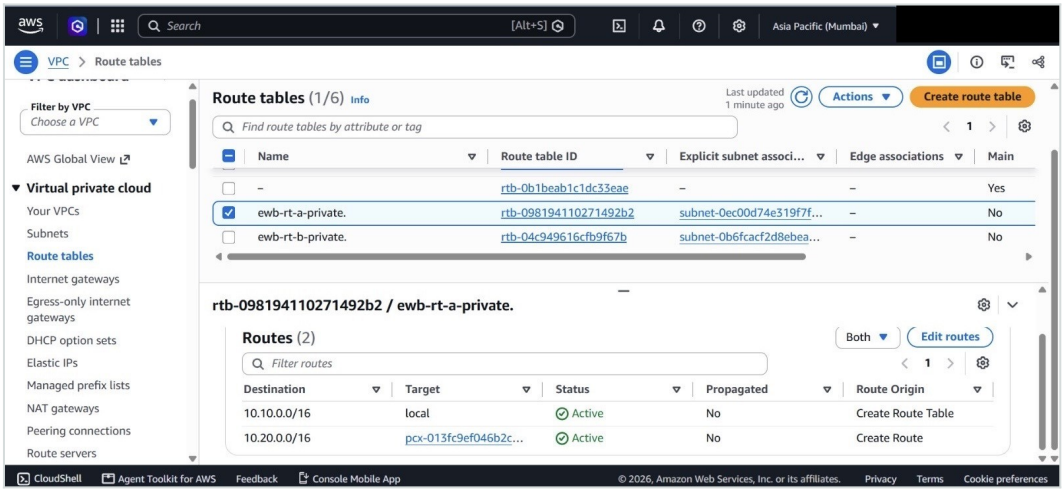


Figure 28 – E4e-alt: Route table ewb-rt-a-private: routes, with the AWS console header.

What it proves: Same routes as E4e. The header shows the Asia Pacific (Mumbai) Region selector (account details masked).

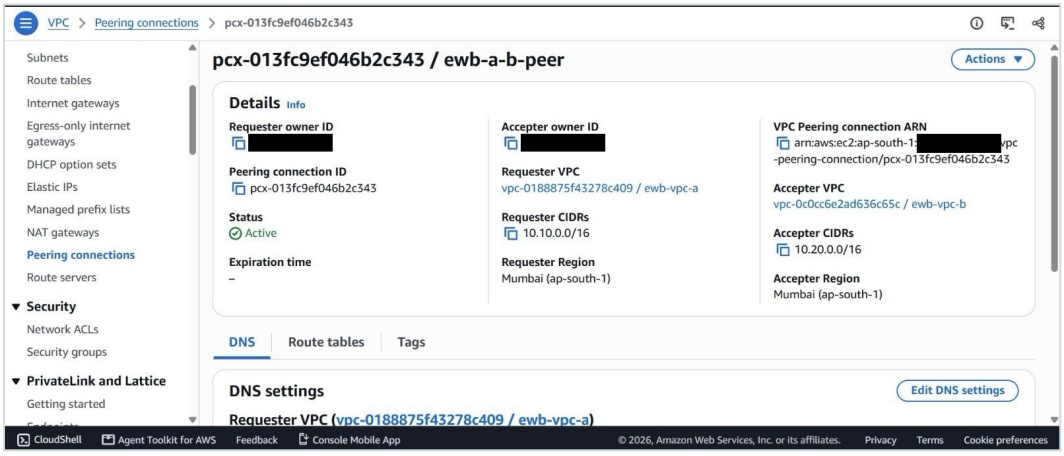


Figure 29 – E7-alt: Peering connection ewb-a-b-peer (second capture).

What it proves: Same details and Active status as E7, captured at a different scroll position.

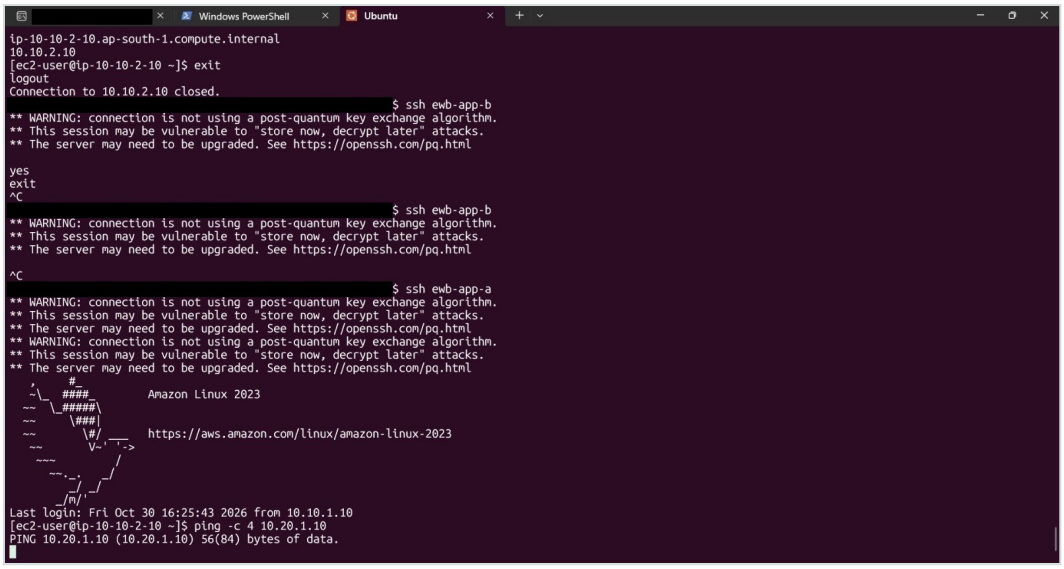


Figure 30 – E8-alt: Terminal on App-A: baseline ping -c 4 10.20.1.10 started and waiting for replies.

What it proves: The ping has printed its header line and no replies yet; the session continues in E8 and ends with 100% packet loss. Above are two ssh ewb-app-b attempts interrupted with Ctrl-C and then a successful ssh ewb-app-a.

13. Final Submission Checklist

☑	Item
☑	All sections of the project guidance completed: overview, architecture, inventory, VPCs/subnets/route tables, peering and security groups, connectivity, controlled failure/recovery, cleanup, learnings, evidence appendix.
☑	Actual VPC, subnet, Internet gateway, route table, security group, peering and instance IDs inserted.
☑	Architecture diagram and network configuration summary included.
☑	Route tables and subnet associations shown (E4a to E4g); peering connection Active (E7).
☑	Baseline failure (E8), App-A to App-B (E13), App-B to App-A (E10) and SSH via bastion (E11) documented with real output.
☑	Controlled ICMP failure (E12) and recovery (E13) documented.
☑	Cleanup evidence included (E14a to E14d).
☑	Account ID, public IP, laptop user name and host-key fingerprint masked; no passwords, keys or access keys shown.
☑	Evidence IDs, figure captions and table of contents checked.
☑	Report saved as AWS_VPC_Peering_Project_Report.pdf (editable DOCX also provided).